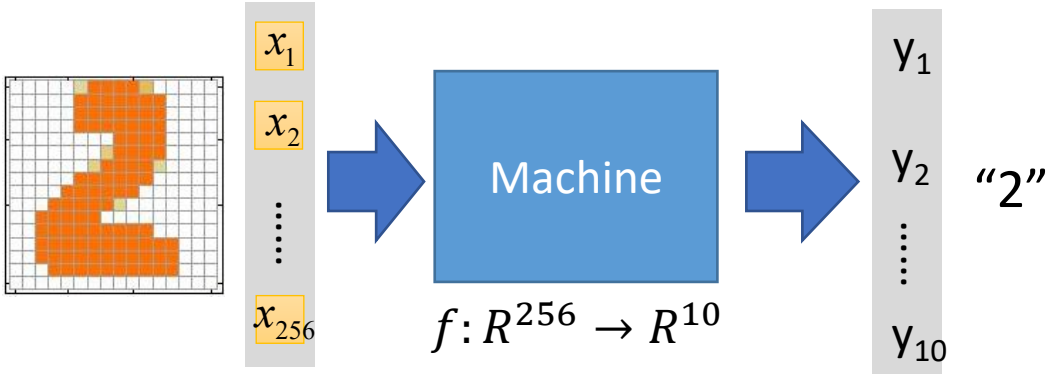
Part 2.2:

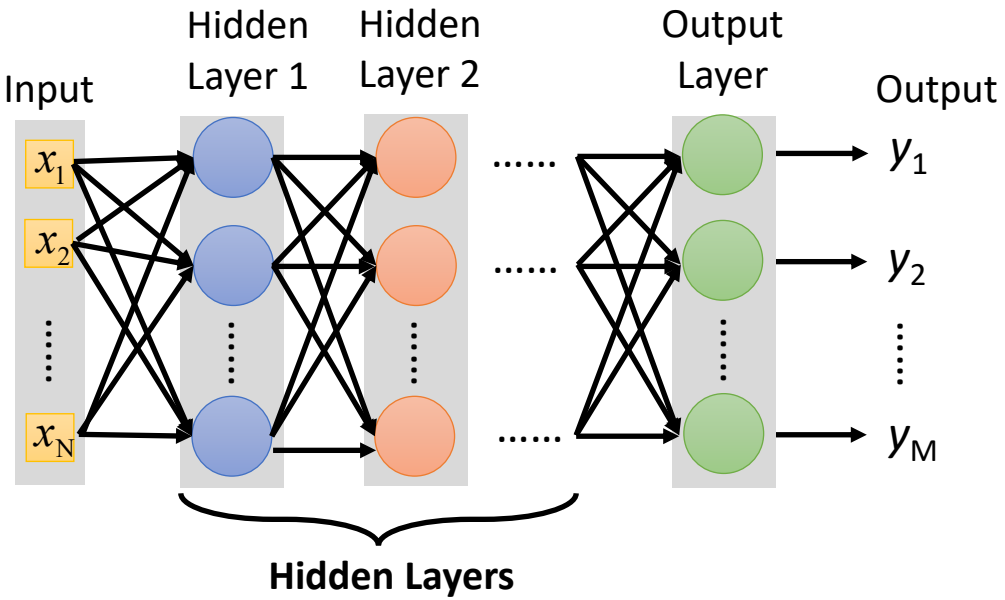
Training Deep Networks

- An example: Handwritten digit recognition

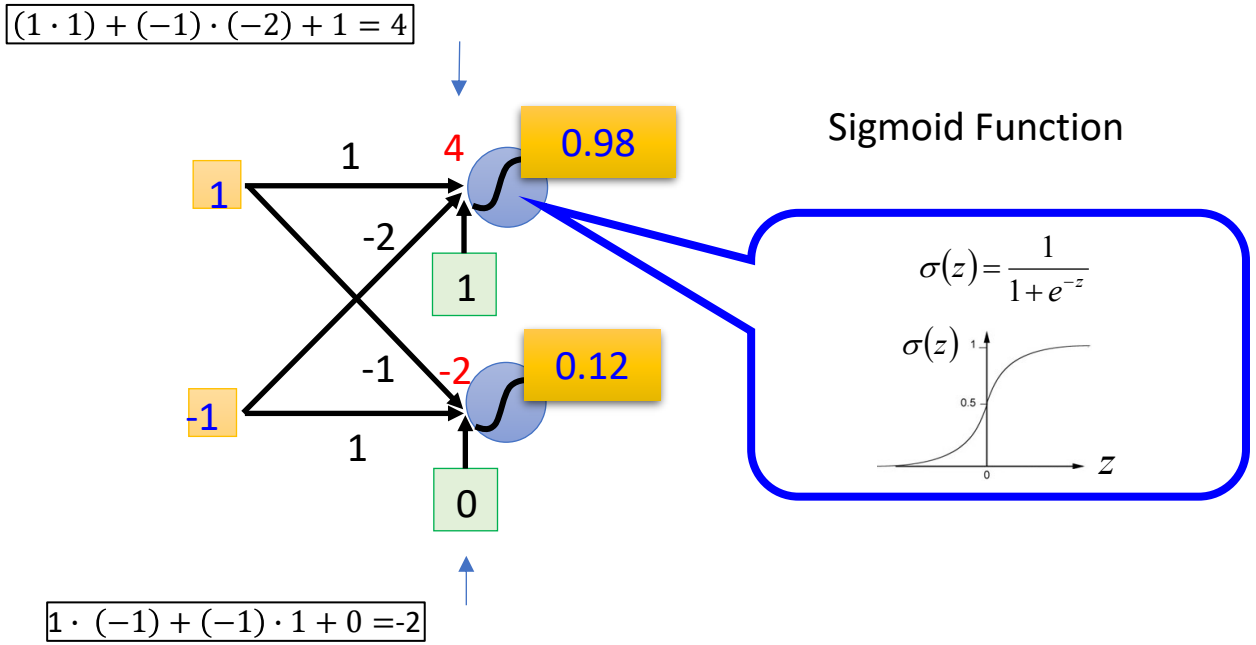


The function f is represented by a neural network

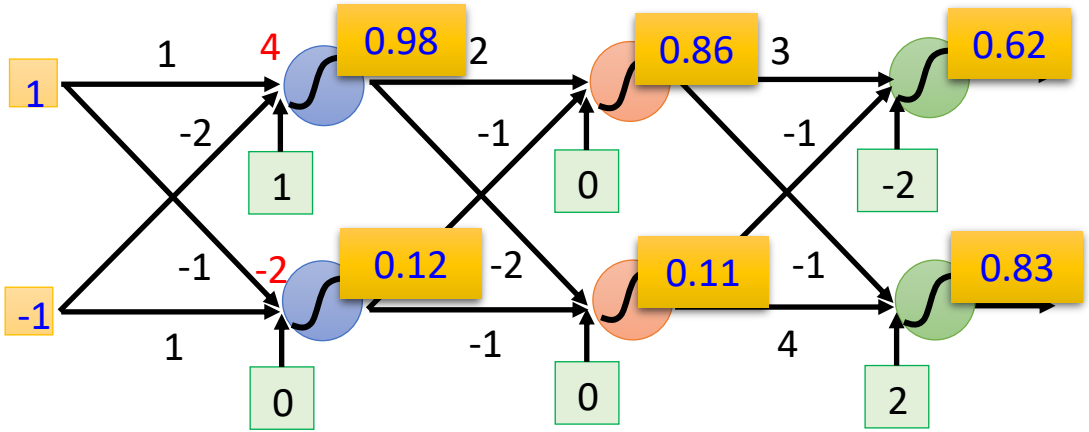
- Multi-Layer Perceptron or MLP



- A simple network, toy example



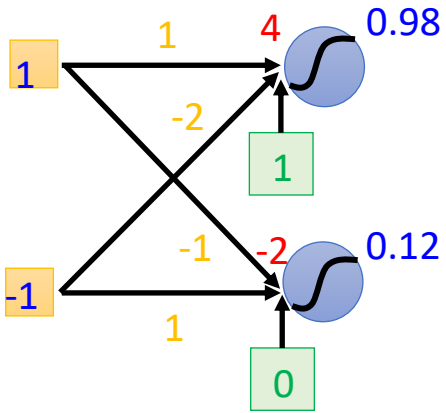
- A simple network, toy example (cont'd)
  - For an input vector $[1 \quad -1]^T$, the output is $[0.62 \quad 0.83]^T$



$$f: R^2 \rightarrow R^2 \qquad f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix}$$

Matrix Operation

- Matrix operations facilitate computations in deep networks, particularly when handling multidimensional inputs and outputs

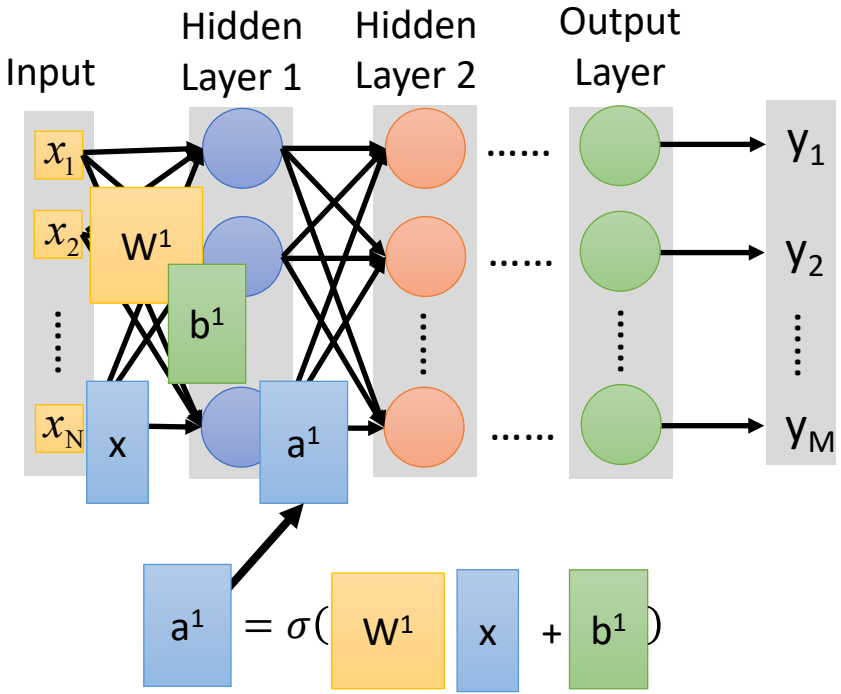


$$\sigma\left(\begin{bmatrix}1 & -2\\ -1 & 1\end{bmatrix}\begin{bmatrix}1\\ -1\end{bmatrix}+\begin{bmatrix}1\\ 0\end{bmatrix}\right)=\begin{bmatrix}0.98\\ 0.12\end{bmatrix}$$

$$\begin{bmatrix}4\\ -2\end{bmatrix}$$

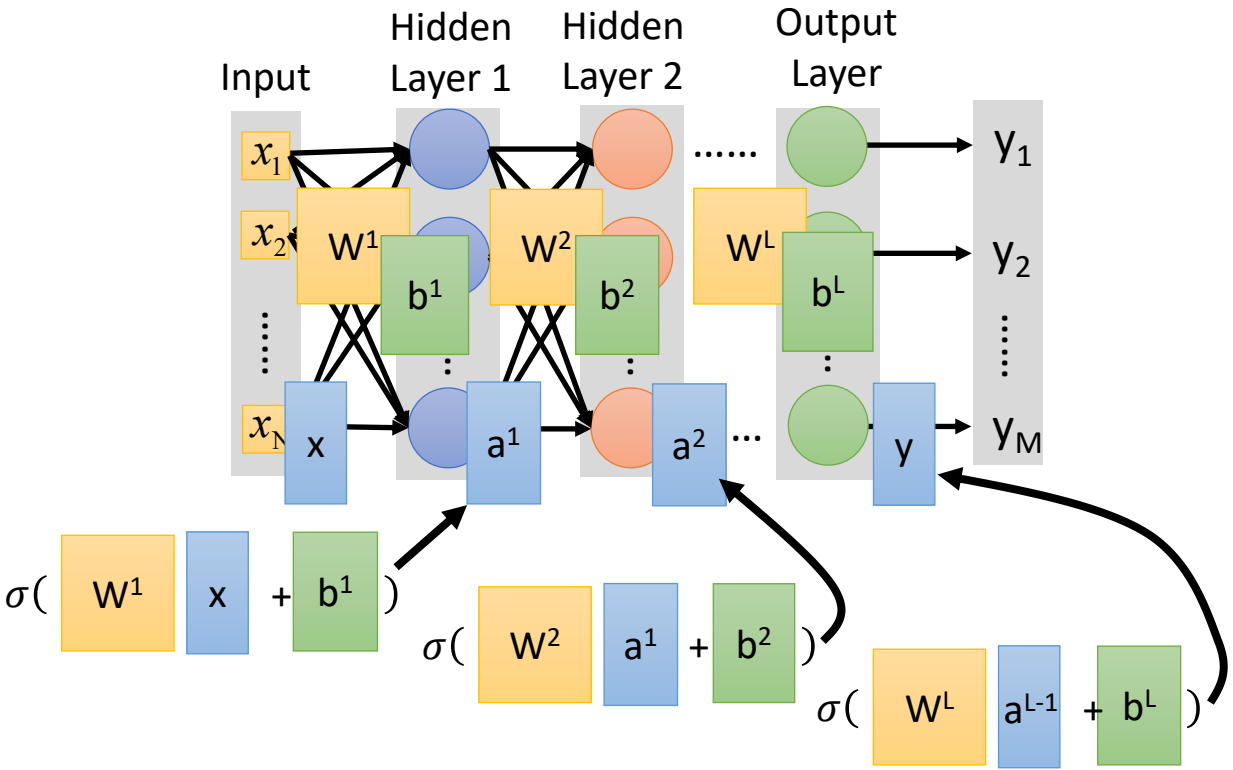
Matrix Operation

- Multilayer NN, matrix calculations for the first layer
  - Input vector x, weights matrix W^1, bias vector b^1, output vector a^1



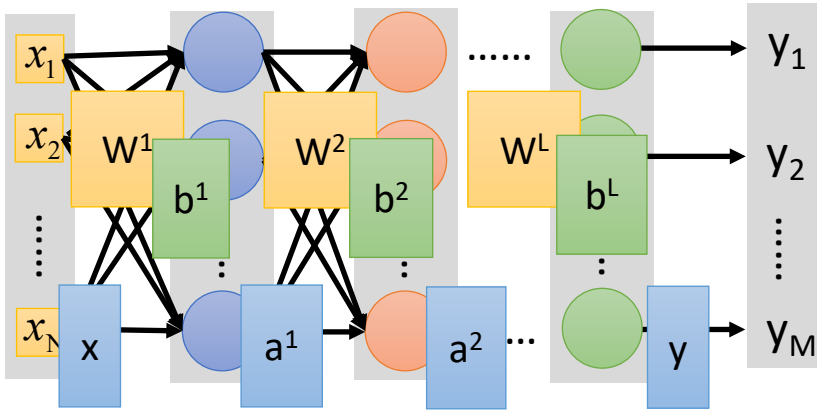
Matrix Operation

- Multilayer NN, matrix calculations for all layers



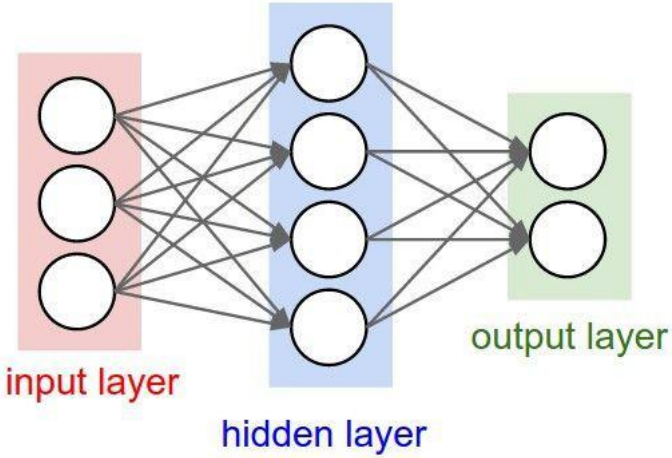
Matrix Operation

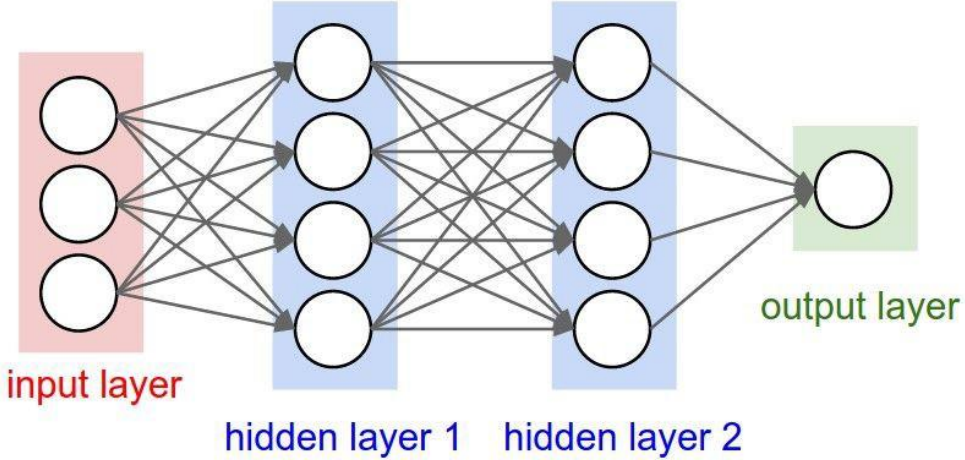
- Multilayer NN, function f maps inputs x to outputs y, i.e., $y = f(x)$



$$\mathbf{y} = f(\mathbf{x}) = \sigma(\mathbf{W}^L \cdots \sigma(\mathbf{W}^2 \sigma(\mathbf{W}^1 \mathbf{x} + \mathbf{b}^1) + \mathbf{b}^2) \cdots + \mathbf{b}^L)$$

Neural network terminology

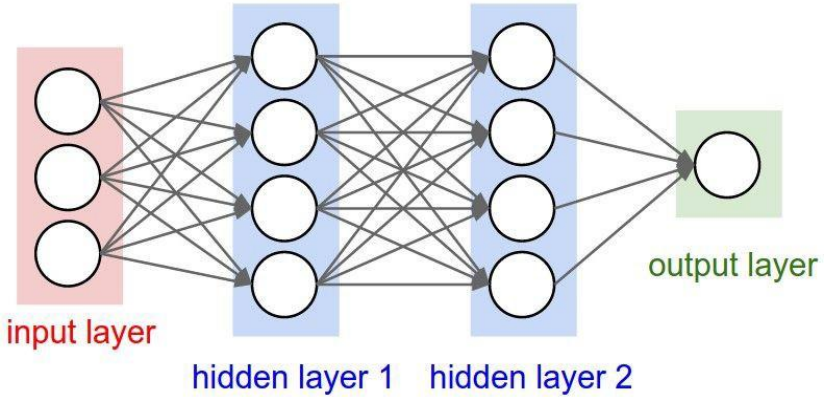




"2-layer Neural Net", or
"1-hidden-layer Neural Net", or
"single-hidden-layer Neural Net"

"3-layer Neural Net", or
"2-hidden-layer Neural Net"

Feed-forward computation




```
import torch

# Define layer dimensions
x_dim, h_dim_1, h_dim_2, y_dim = 3, 4, 4, 1

# Define linear layers
linear_1 = torch.nn.Linear(x_dim, h_dim_1)  # R^3 → R^4
linear_2 = torch.nn.Linear(h_dim_1, h_dim_2)  # R^4 → R^4
linear_3 = torch.nn.Linear(h_dim_2, y_dim)  # R^4 → R^1

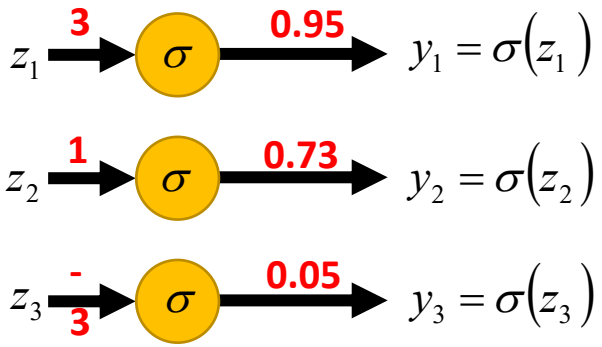
# Create a random input tensor
x = torch.randn(1, x_dim)  # Shape = [1, 3]

# Forward pass through layers
h1 = linear_1(x)  # Shape = [1, 4]
h2 = linear_2(h1)  # Shape = [1, 4]
y = linear_3(h2)  # Shape = [1, 1]
```

Softmax Layer

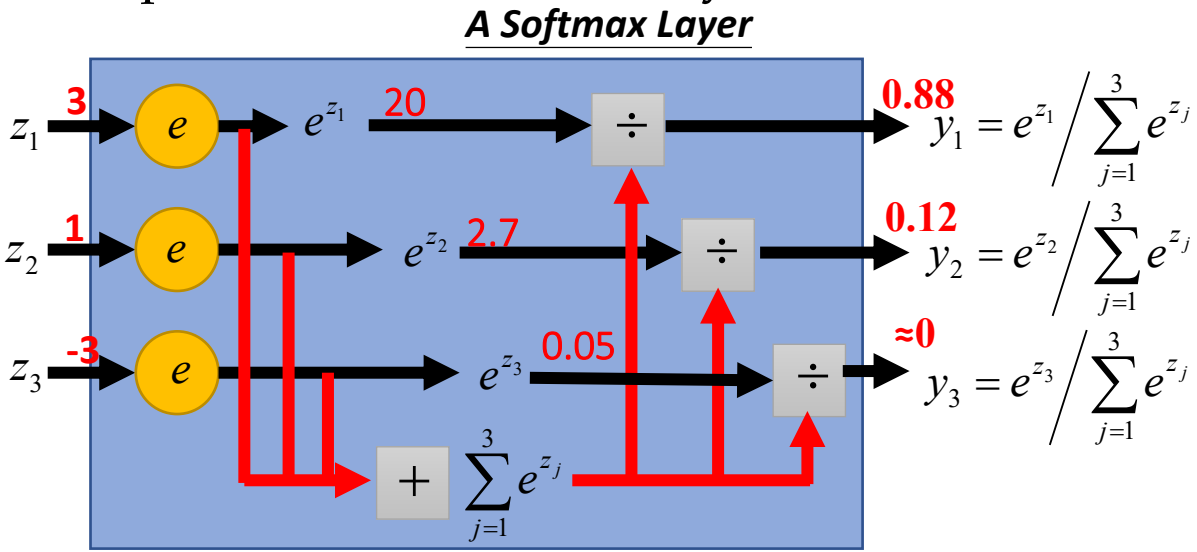
- In multi-class classification tasks, the output layer is typically a ***softmax layer***
  - i.e., it employs a ***softmax activation function***
  - If a layer with a sigmoid activation function is used as the output layer instead, the predictions by the NN may not be easy to interpret, i.e., the outputs don't sum up to 1
    - Note that an output layer with sigmoid activations can still be used for binary classification

A Layer with Sigmoid Activations



Softmax Layer

- A Softmax layer applies the Softmax function to convert logits into probability values ranging from [0, 1]
- The outputs sum to 1, making it suitable for multi-class classification
- The values z inputted to the softmax layer are referred to as *logits*



Probability:

- $0 < y_i < 1$
- $\sum_i y_i = 1$

Softmax Layer

```
>>> import torch

# Softmax converts values into probabilities
>>> x = torch.tensor([3, 1, -3], dtype=float)

>>> torch.nn.Softmax()(x)
[0.8789, 0.1189, 0.0022]

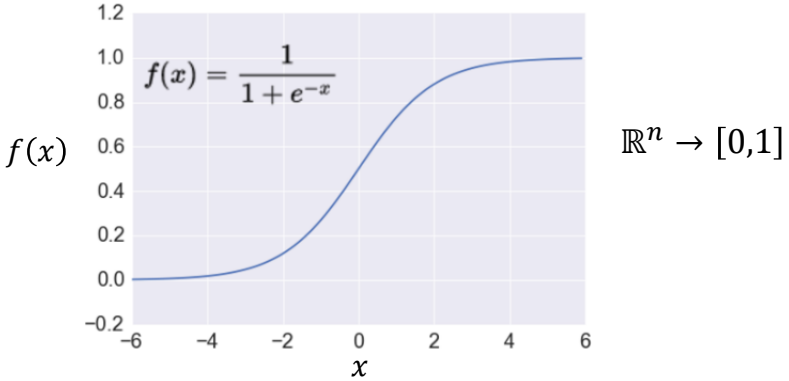
# Softmax always sums to 1.0
>>> torch.sum(torch.nn.Softmax()(x))
1.
```

Probability:

- $0 < y_i < 1$
- $\sum_i y_i = 1$

Activation: Sigmoid

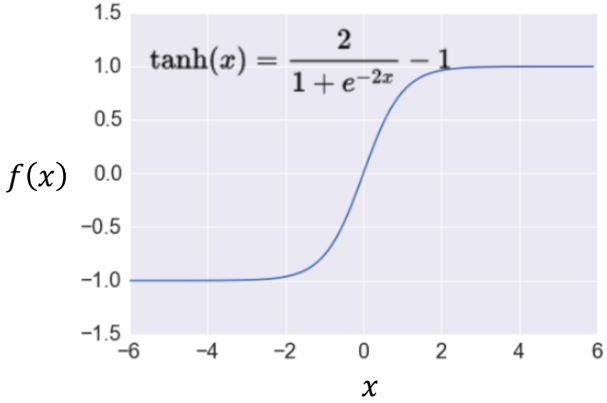
- ***Sigmoid function*** σ: takes a real-valued number and "squashes" it into the range between 0 and 1
  - The output can be interpreted as the firing rate of a biological neuron
    - Not firing = 0; Fully firing = 1
  - When the neuron's activation are 0 or 1, sigmoid neurons saturate
    - Gradients at these regions are almost zero (almost no signal will flow)
  - Sigmoid activations are less common in modern NNs



$\mathbb{R}^n \rightarrow [0,1]$

Activation: Tanh

- ***Tanh function***: takes a real-valued number and "squashes" it into range between -1 and 1
  - Like sigmoid, tanh neurons saturate
  - Unlike sigmoid, the output is zero-centered
    - It is therefore preferred to sigmoid
  - Tanh is a scaled sigmoid: $\tanh(x) = 2 \cdot \sigma(2x) - 1$



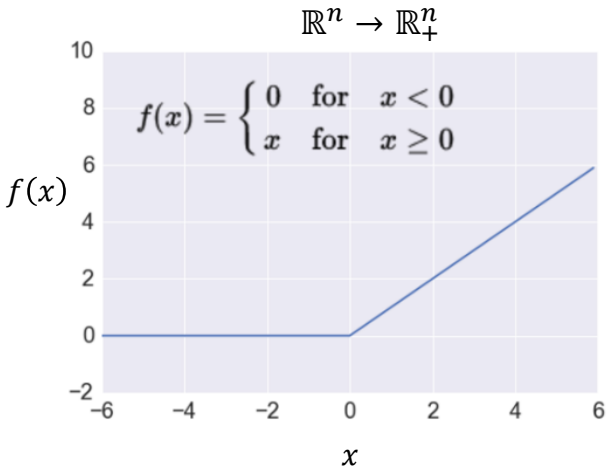
$\mathbb{R}^n \rightarrow [-1,1]$

Activation: ReLU

- ***ReLU*** (Rectified Linear Unit): takes a real-valued number and thresholds it at zero

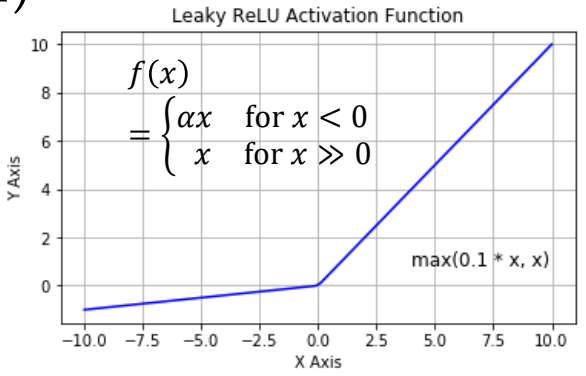
$$f(x) = \max(0, x)$$

  - Most modern deep NNs use ReLU activations
  - ReLU is fast to compute
    - Compared to sigmoid, tanh
    - Simply threshold a matrix at zero
  - Accelerates the convergence of gradient descent
    - Due to linear, non-saturating form
  - Prevents the gradient vanishing problem



Activation: Leaky ReLU

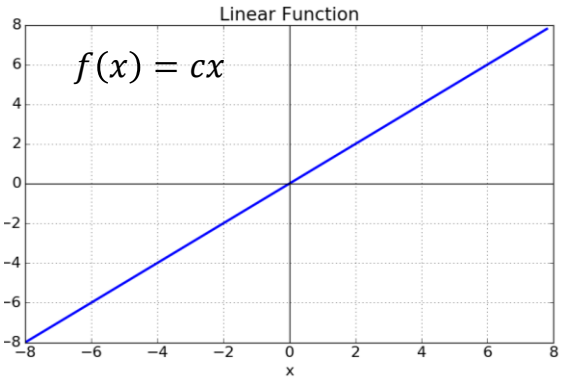
- The problem of ReLU activations: they can "die"
  - ReLU could cause weights to update in a way that the gradients can become zero and the neuron will not activate again on any data
  - E.g., when a large learning rate is used
- ***Leaky ReLU*** activation function is a variant of ReLU
  - Instead of the function being 0 when $x < 0$, a leaky ReLU has a small positive slope (e.g., $\alpha = 0.01$, or similar)
  - This resolves the dying ReLU problem
  - Most current works still use ReLU
    - With a proper setting of the learning rate, the problem of dying ReLU can be avoided



Activation: Linear Function

- ***Linear function*** means that the output signal is proportional to the input signal to the neuron
  - If the value of the constant c is 1, it is also called identity activation function
  - This activation type is used in regression problems
    - E.g., the last layer can have linear activation function, in order to output a real number (and not a class membership)

$$\mathbb{R}^n \rightarrow \mathbb{R}^n$$

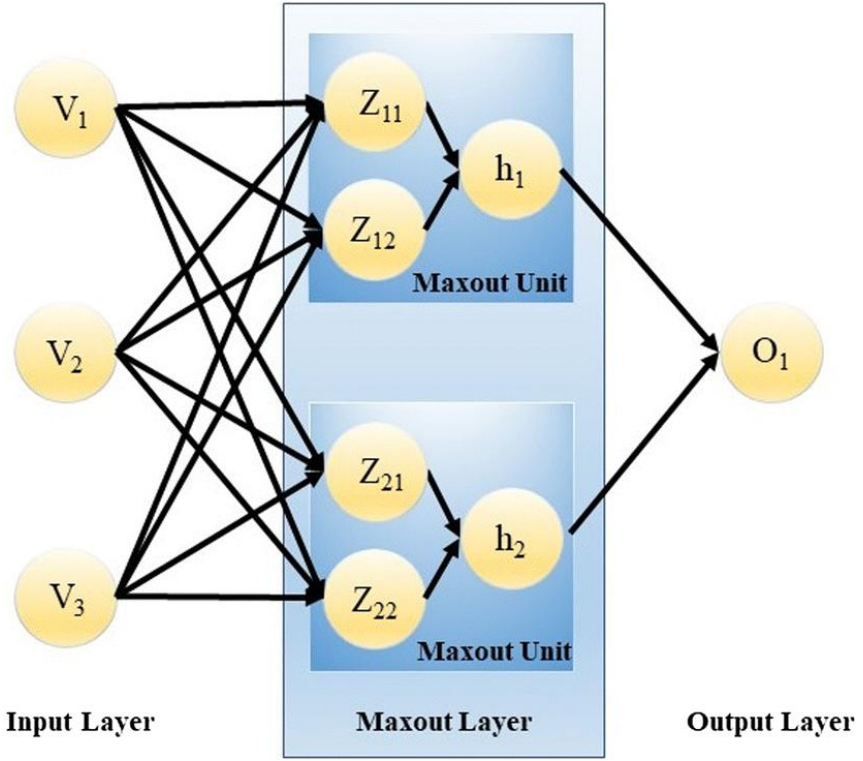


Other Activation Functions

- *For example,*

- Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

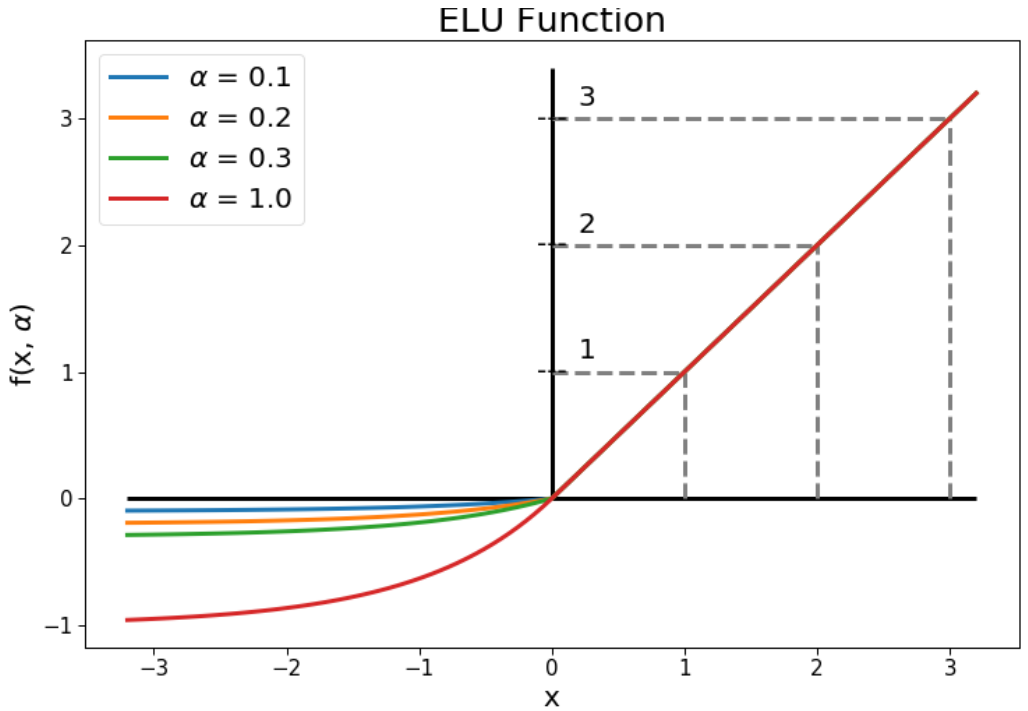


2 Maxout Neurons

Other Activation Functions

- Exponential Linear Unit (ELU)
  - Non-zero derivative for both positive and negative input (no more dying ReLU)

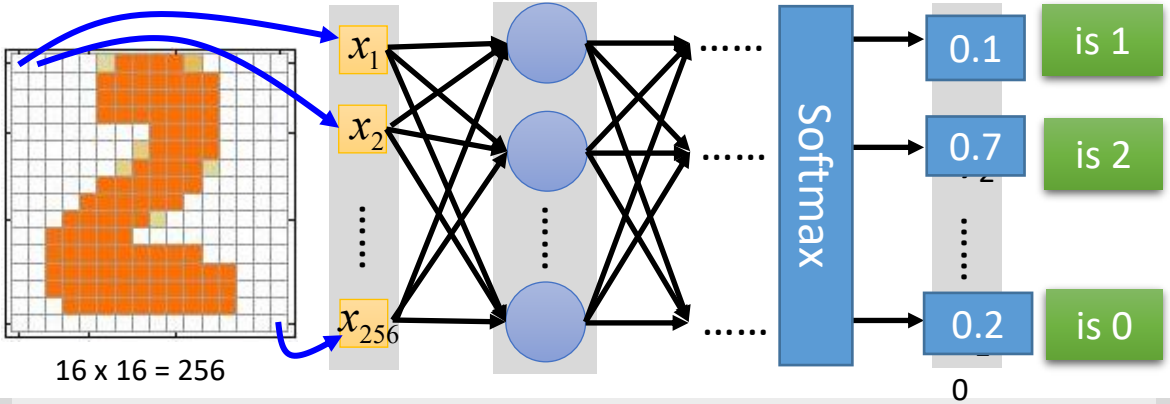
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Training NNs for Classification

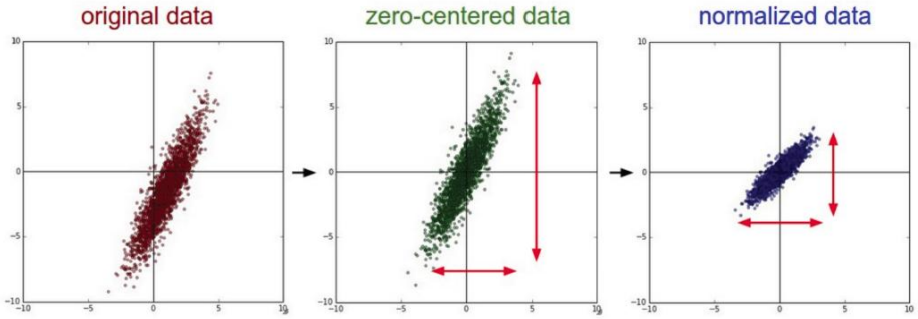
- The network ***parameters*** θ include the weight matrices and bias vectors from all layers
  - Often, the model parameters θ are referred to as weights
- Training a model to learn a set of parameters θ that are optimal (according to a criterion) is one of the greatest challenges in ML

$$\theta = \{W^1, b^1, W^2, b^2, \cdots W^L, b^L\}$$



Training NNs for Classification

- *Data preprocessing* - helps convergence during training
  - Standardization: Divide each feature by its standard deviation + subtract mean
    - standard deviation 1, mean 0 for each data dimension (feature)
  - Normalization: (x - x_min)/(x_max - x_min)
    - Scaled data to [0,1] range



Data Preprocessing

```
>>> import torch
>>> x = torch.randn([100])
>>> print(x.mean().item())
0.02

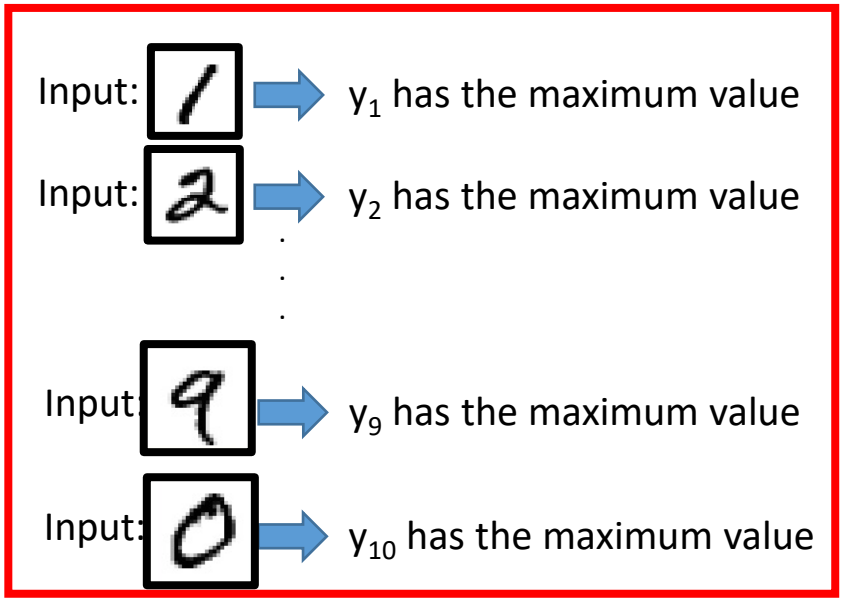
>>> print(x.var().item())
0.9

# Standardization
>>> x_standardized = (x - x.mean()) / x.std()
>>> print(x_standardized.mean(), x_standardized.var())
0.0, 1.0

# Normalization
>>> x_normalized = (x - x.min()) / (x.max() - x.min())
>>> print(x_normalized.min(), x_normalized.max())
0.0, 1.0
```

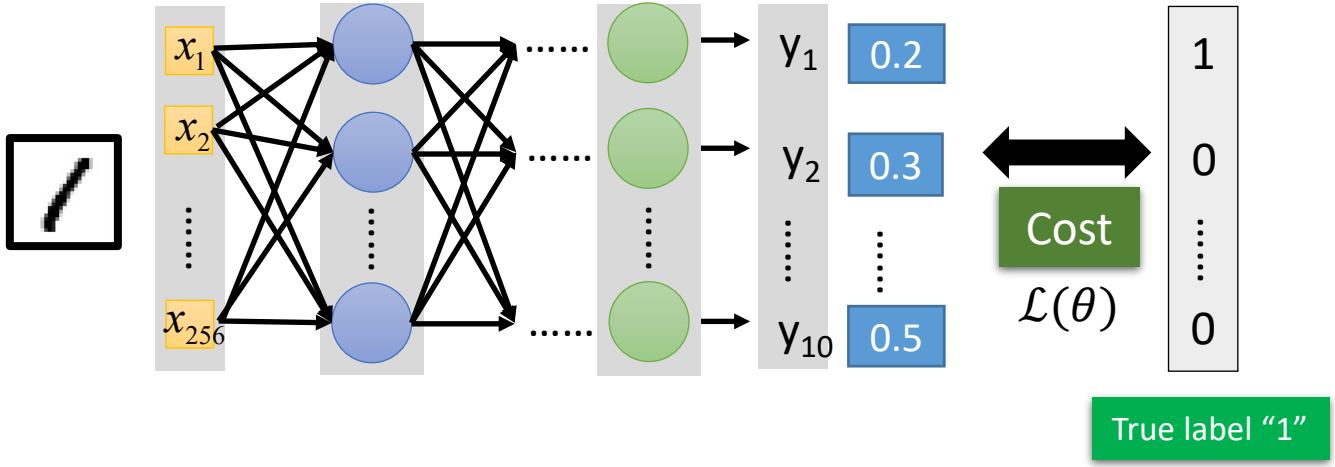
Training NNs for Classification

- To train a NN, set the parameters θ such that for a training subset of images, the corresponding elements in the predicted output have maximum values



Training NNs for Classification

- Define a ***loss function***/objective function/cost function $\mathcal{L}(\theta)$ that calculates the difference (error) between the model prediction and the true label
  - E.g., $\mathcal{L}(\theta)$ can be mean-squared error, cross-entropy, etc.



To Train a Multi-layer Perceptron (MLP) as a Classifier

- If there are K classes
- Use K output neurons
- Let the target output for class k be $(o_1, o_2, \ldots, o_k, \ldots, o_K)$, with only $o_k = 1$ and all other output = 0
- After training and during testing, if a test pattern produces an output $(o_1, o_2, \ldots, o_k, \ldots, o_K)$, with o_k being the greatest among all o's, then this test pattern should be classified as class k
- This is mathematically proven to be an **optimal classifier** with minimum error, if the neural network is trained to a global optimum using either cross-entropy or mean squared error loss function

Training NNs for Classification

```
>>> import torch
>>> from torch.nn.functional import cross_entropy
```

P(y=0) P(y=3)

```
>>> y = torch.tensor([0.8, 0.1, 0.06, 0.04])

>>> cross_entropy(y, torch.tensor(0))
0.8926

>>> cross_entropy(y, torch.tensor(1))
1.5926

>>> cross_entropy(y, torch.tensor(2))
1.6326

>>> cross_entropy(y, torch.tensor(3))
1.6526
```

CE Loss if y = 0

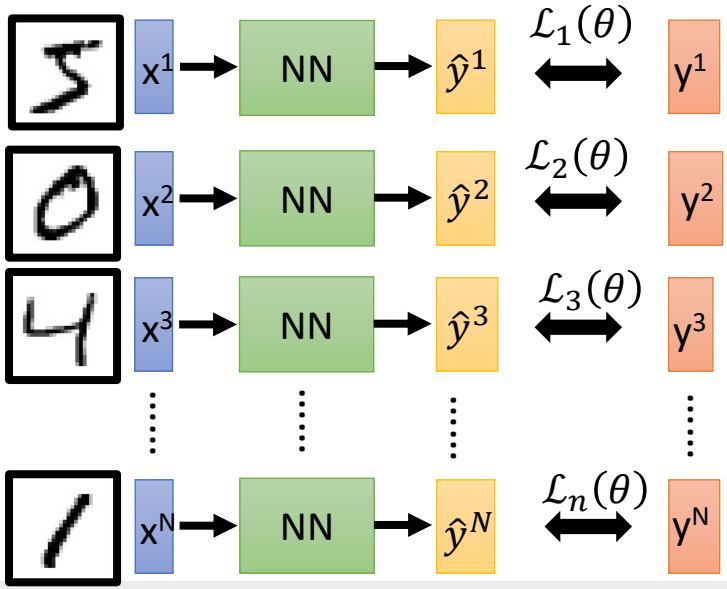
CE Loss if y = 1

CE Loss if y = 2

CE Loss if y = 3

Training NNs for Classification

- For a training set of N images, calculate the total loss overall all images: $\mathcal{L}(\theta) = \sum_{n=1}^{N} \mathcal{L}_n(\theta)$
- Find the optimal parameters θ^* that minimize the total loss $\mathcal{L}(\theta)$



Loss Functions

Training examples

Pairs of N inputs x_i and ground-truth class labels y_i

Loss function

Cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K}\left[y_k^{(i)}\log\hat{y}_k^{(i)} + \left(1-y_k^{(i)}\right)\log\left(1-\hat{y}_k^{(i)}\right)\right]$$

Ground-truth class labels y_i and model predicted class labels $\hat{y}_i$

Loss Functions

Training examples

Pairs of N inputs x_i and ground-truth output values y_i

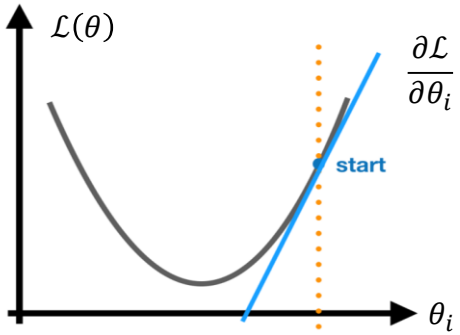
Loss function

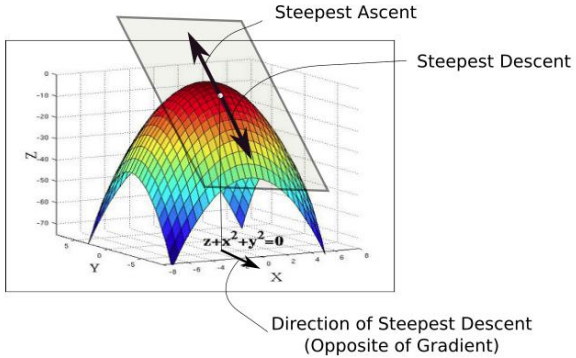
Mean Squared Error

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2$$

Training NNs

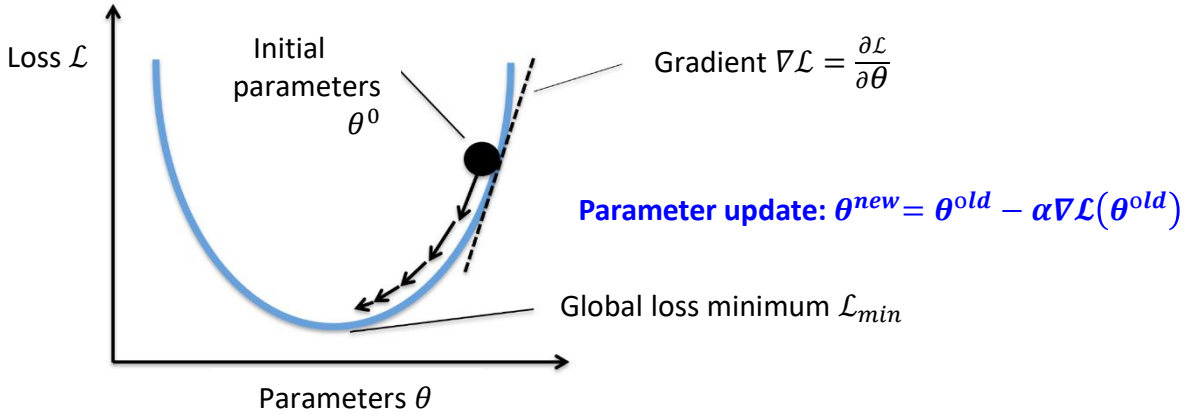
- Optimizing the loss function $\mathcal{L}(\theta)$
  - Almost all DL models these days are trained with a variant of the ***gradient descent*** (GD) algorithm
  - GD applies iterative refinement of the network parameters θ
  - GD uses the opposite direction of the gradient of the loss with respect to the NN parameters (i.e.,$\nabla\mathcal{L}(\theta) = [\partial\mathcal{L}/\partial\theta_i]$) for updating θ
    - The gradient of the loss function $\nabla\mathcal{L}(\theta)$ gives the direction of fastest increase of the loss function $\mathcal{L}(\theta)$ when the parameters θ are changed





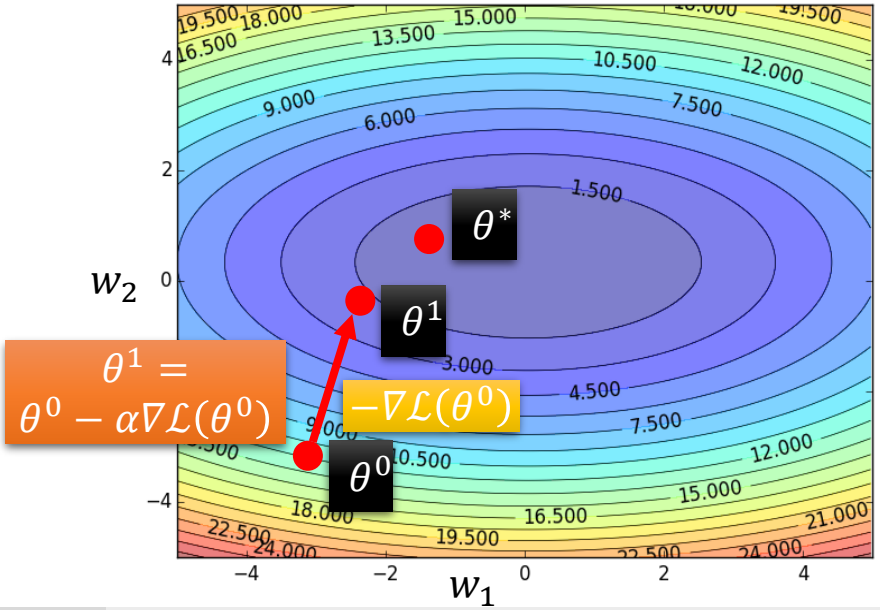
Gradient Descent Algorithm

- Steps in the ***gradient descent algorithm***:
  1. Initialization: Randomly initialize the model parameters, θ^0. Let $\theta^{old} = \theta^0$
  2. Compute the gradient of the loss function at θ^{old} : $\nabla\mathcal{L}(\theta^{old})$
  3. Update the parameters as: $\theta^{new} = \theta^{old} - \alpha\nabla\mathcal{L}(\theta^{old})$
     - Where α is the learning rate
  4. Let $\theta^{old} = \theta^{new}$. Go to step 2 and repeat (until a terminating criterion is reached)



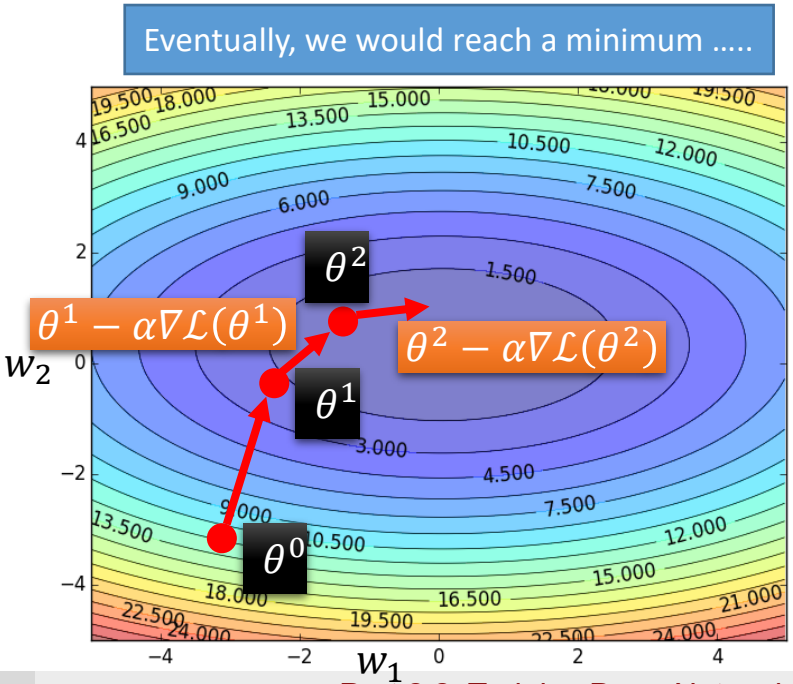
Gradient Descent Algorithm

- Example: a NN with only 2 parameters w_1 and w_2, i.e., $\theta = \{w_1, w_2\}$
  - The different colors represent the values of the loss (minimum loss θ^* is ≈ 1.3)



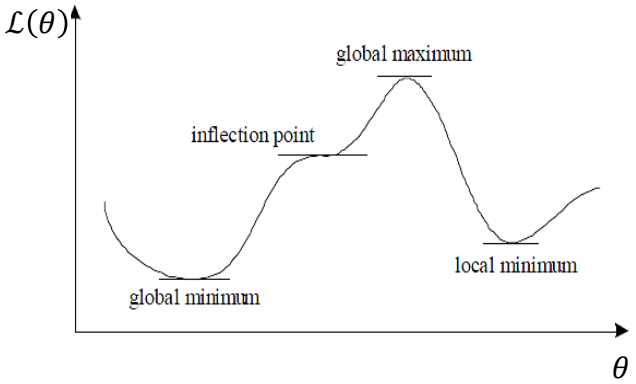
Gradient Descent Algorithm

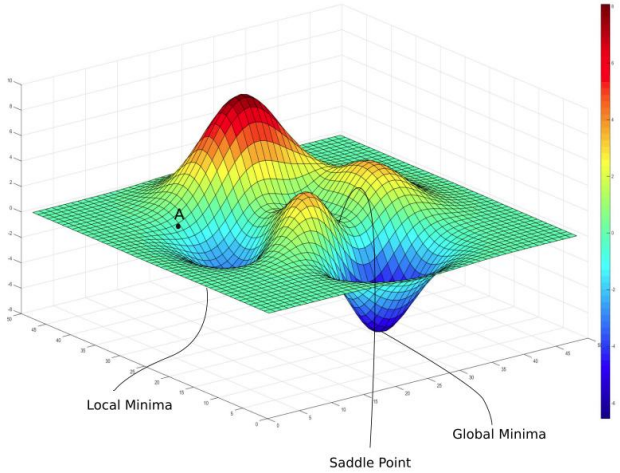
- Example (contd.)



Gradient Descent Algorithm

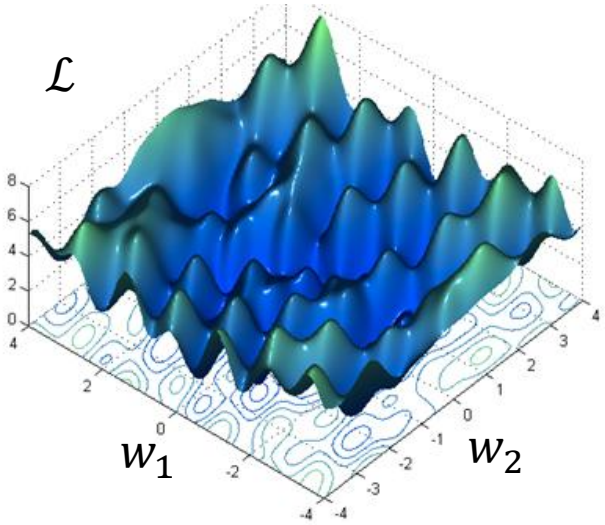
- Gradient descent algorithm stops when a local minimum of the loss surface is reached
  - GD does not guarantee reaching a global minimum
  - However, empirical evidence suggests that GD works well for NNs





Gradient Descent Algorithm

- For most tasks, the loss surface $\mathcal{L}(\theta)$ is highly complex (and non-convex)
  - Random initialization in NNs results in different initial parameters θ^0 every time the NN is trained
    - Gradient descent may reach different minima at every run
    - Therefore, NN will produce different predicted outputs
  - In addition, currently we don't have algorithms that guarantee reaching a global minimum for an arbitrary loss function



Backpropagation (short for back error propagation)

- Forward propagation (forward pass) refers to passing the inputs x through the hidden layers to obtain the model outputs (predictions) y
  - The loss $\mathcal{L}(y, \hat{y})$ function is then calculated
  - Backpropagation traverses the network in reverse order, from the outputs y backward toward the inputs x to calculate the gradients of the loss $\nabla\mathcal{L}(\theta)$
  - The chain rule is used for calculating the partial derivatives of the loss function with respect to the parameters θ in the different layers in the network
- Each update of the model parameters θ during training takes one forward and one backward pass (e.g., of a batch of input data samples)
- Automatic calculation of the gradients (automatic differentiation) is available in all current deep learning libraries
  - It significantly simplifies the implementation of deep learning algorithms, since it obviates deriving the partial derivatives of the loss function by hand

Barebones PyTorch Backpropagation

```
import torch

# Define our dataset, the XOR problem
X = torch.tensor([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=torch.float)
Y = torch.tensor([0, 1, 1, 0], dtype=int)

# We use a Linear Network
network = torch.nn.Linear(2, 2)                              # Our network
loss_fn = torch.nn.CrossEntropyLoss()                        # Our Loss
optim   = torch.optim.SGD(network.parameters(), lr=0.1)      # Our Optimizer
epochs  = 50                                                 # No. Epochs

# Training Loop
for epoch in range(epochs):
    for x, y in zip(X, Y):
        optim.zero_grad()           # Reset Gradients
        y_pred = network(x)         # Forward Propagate x
        loss = loss_fn(y_pred, y)   # Evaluate Loss
        loss.backward()             # Backpropagate x
        optimizer.step()            # Update Weights with Optimizer

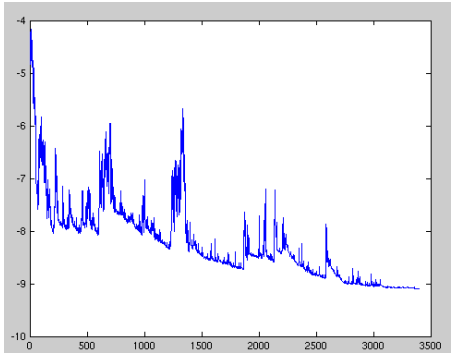
>>> print(loss)
0.7437
```

Mini-batch Gradient Descent

- It is *wasteful* to compute the loss over the entire training dataset to perform a single parameter update for large datasets
  - E.g., ImageNet has 14M images
  - Therefore, GD (a.k.a. vanilla GD) is almost always replaced with mini-batch GD
- ***Mini-batch gradient descent***
  - Approach:
    - Compute the loss $\mathcal{L}(\theta)$ on a mini-batch of images, update the parameters θ, and repeat until all images are used
    - At the next epoch, shuffle the training data, and repeat the above process
  - Mini-batch GD results in much faster training
  - Typical mini-batch size: 32 to 256 images
  - It works because the gradient from a mini-batch is a good approximation of the gradient from the entire training set

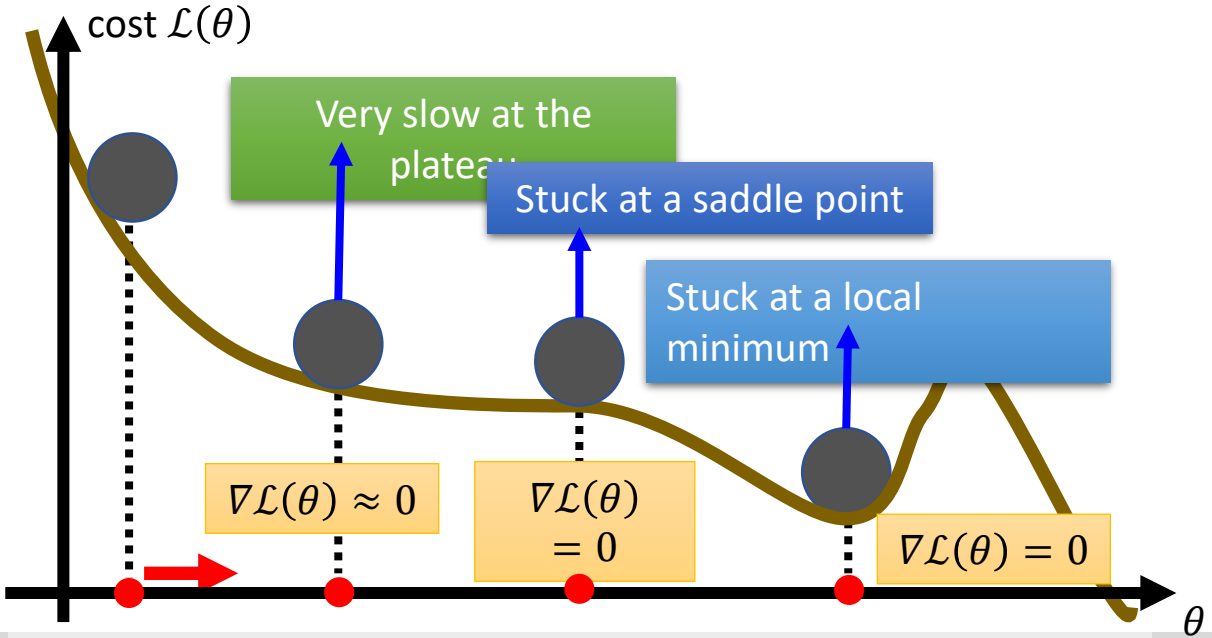
Stochastic Gradient Descent (SGD)

- SGD uses mini-batches that consist of a single input example
  - E.g., one image mini-batch
- Although this method is very fast, it may cause significant fluctuations in the loss function
  - Therefore, it is less commonly used, and mini-batch GD is preferred
- In most DL libraries, SGD typically means a mini-batch GD (with an option to add momentum)



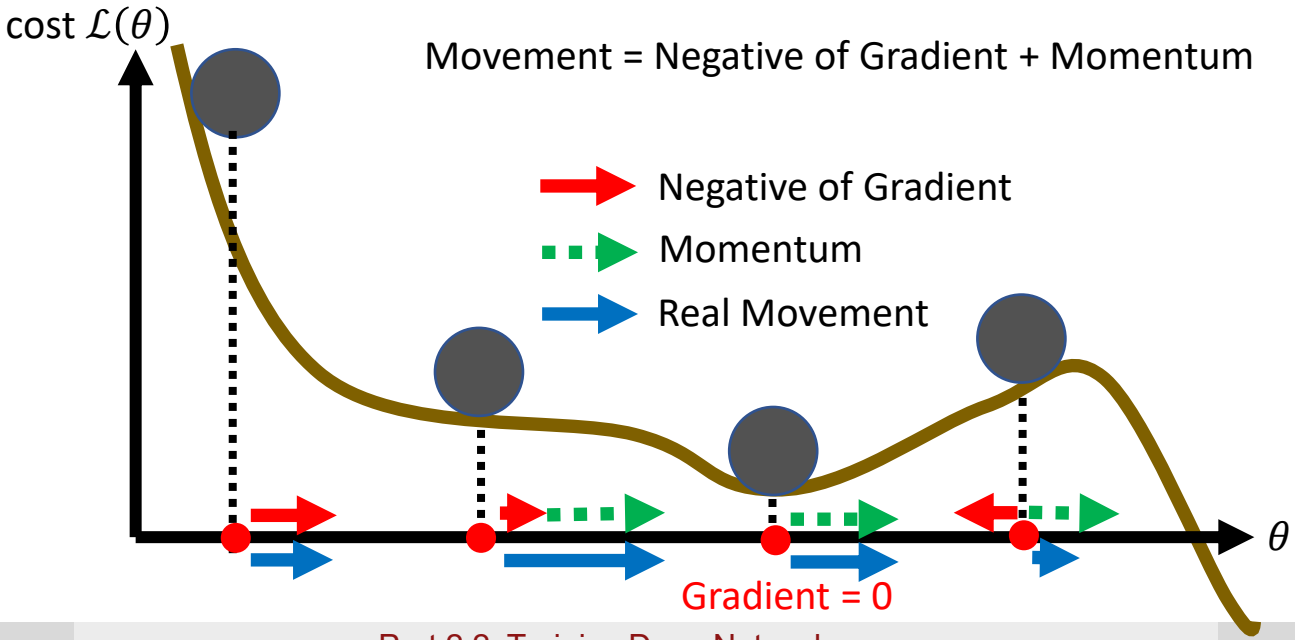
Problems with Gradient Descent

- Besides the local minima problem, the GD algorithm can be very slow at plateaus, and it can get stuck at saddle points



Gradient Descent with Momentum

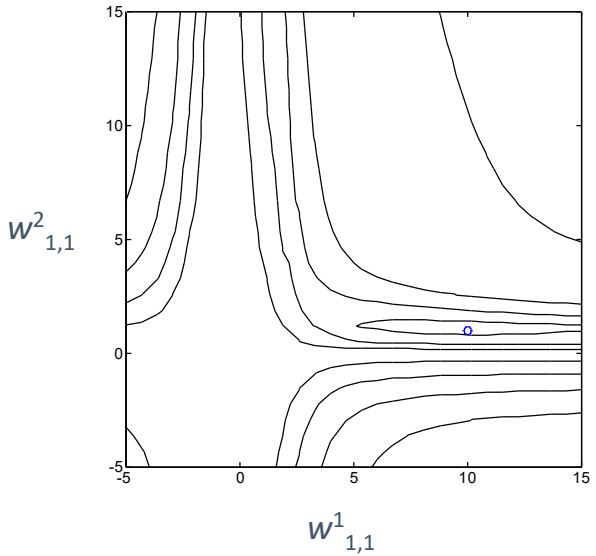
- ***Gradient descent with momentum*** uses the momentum of the gradient for parameter optimization

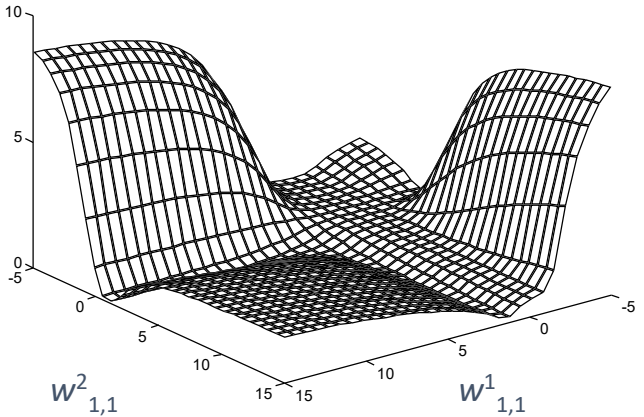


Example:

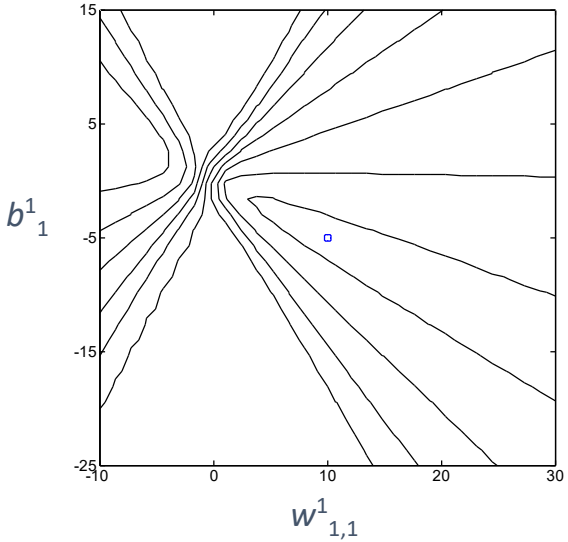
a 1-2-1 network (1 input, 2 hidden neurons, 1 output neuron; 4 weights, 3 biases, i.e., 7 parameters)

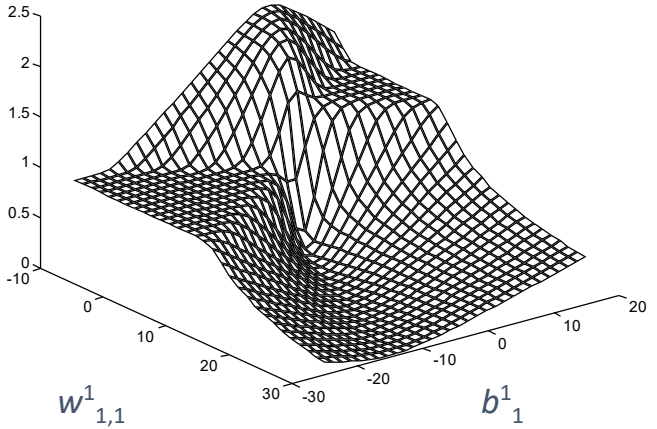
Squared Error vs. $w^1_{1,1}$ and $w^2_{1,1}$



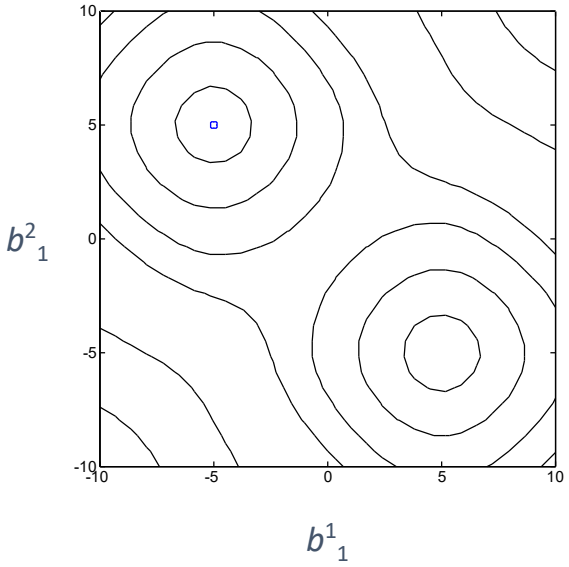


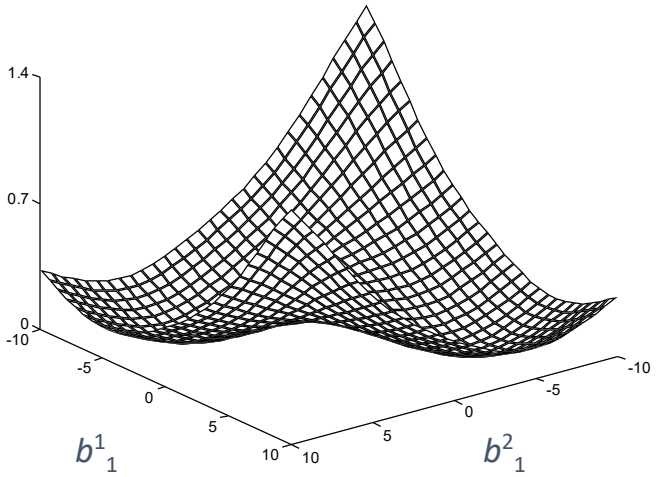
Squared Error vs. $w^1_{1,1}$ and b^1_1



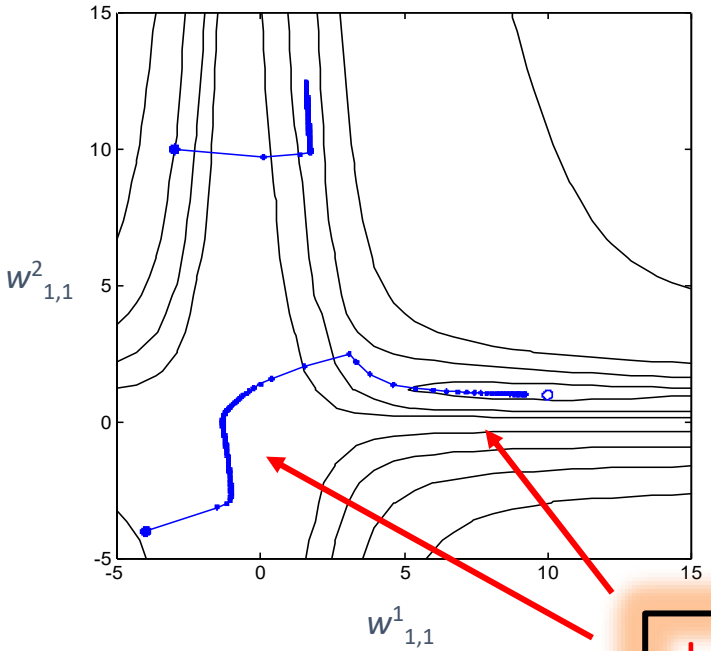


Squared Error vs. b^1_1 and b^1_2

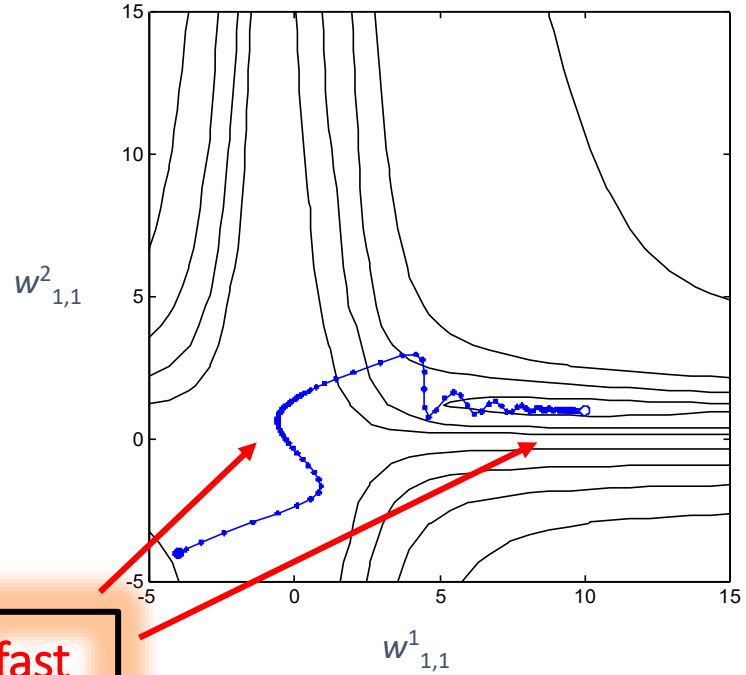




GD (without momentum)



GD with momentum



```
# Sample Python Program for a 1-2-1 neural network, trained with BP vs BP + momentum
# The input data consists of two samples, [0] and [1]
# The corresponding output data is the same as the input data, [0] and [1]
# Prints the loss (mean squared error) for every 1000 epochs

import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, learning_rate, momentum=None):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = learning_rate
        self.momentum = momentum

        # Initialize weights and biases randomly
        self.weights_input_hidden = np.random.randn(self.input_size, self.hidden_size)
        self.bias_hidden = np.zeros((1, self.hidden_size))
        self.weights_hidden_output = np.random.randn(self.hidden_size, self.output_size)
        self.bias_output = np.zeros((1, self.output_size))
```

```
# Initialize velocities for momentum
        if self.momentum:
            self.velocity_input_hidden = np.zeros((self.input_size, self.hidden_size))
            self.velocity_hidden_output = np.zeros((self.hidden_size, self.output_size))

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, x):
        return x * (1 - x)

    def forward_pass(self, X):
        # Forward pass through the network
        self.hidden_output = self.sigmoid(np.dot(X, self.weights_input_hidden) + self.bias_hidden)
        self.output = self.sigmoid(np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output)
        return self.output
```

```
def backward_pass(self, X, y, output):
    # Backward pass through the network
    error = y - output
    output_delta = error * self.sigmoid_derivative(output)          # *: This denotes element-wise multiplication

    #matrix multiplication between output_delta and the transpose of self.weights_hidden_output:
    error_hidden = output_delta.dot(self.weights_hidden_output.T)
    hidden_delta = error_hidden * self.sigmoid_derivative(self.hidden_output)

    # Update weights and biases
    if self.momentum:
        self.velocity_input_hidden = self.momentum * self.velocity_input_hidden + X.T.dot(hidden_delta) * self.learning_rate
        self.velocity_hidden_output = self.momentum * self.velocity_hidden_output + self.hidden_output.T.dot(output_delta) *
self.learning_rate

        self.weights_input_hidden += self.velocity_input_hidden
        self.weights_hidden_output += self.velocity_hidden_output
    else:
        self.weights_hidden_output += self.hidden_output.T.dot(output_delta) * self.learning_rate
        self.weights_input_hidden += X.T.dot(hidden_delta) * self.learning_rate

    self.bias_output += np.sum(output_delta, axis=0, keepdims=True) * self.learning_rate
    self.bias_hidden += np.sum(hidden_delta, axis=0, keepdims=True) * self.learning_rate
```

```
def train(self, X, y, epochs):
    for epoch in range(epochs):
        # Forward pass
        output = self.forward_pass(X)

        # Backward pass
        self.backward_pass(X, y, output)

        # Print loss for every 1000 epochs
        if epoch % 1000 == 0:
            loss = np.mean(np.square(y - output))
            print(f'Epoch {epoch}, Loss: {loss:.4f}')
```

```
# Example usage without momentum
if __name__ == "__main__":
    # Define input data and labels
    X = np.array([[0], [1]])
    y = np.array([[0], [1]])

    # Initialize and train the neural network without momentum
    nn_no_momentum = NeuralNetwork(input_size=1, hidden_size=2, output_size=1, learning_rate=0.1)
    print("Training without momentum:")
    nn_no_momentum.train(X, y, epochs=10000)

# Example usage with momentum
if __name__ == "__main__":
    # Define input data and labels
    X = np.array([[0], [1]])
    y = np.array([[0], [1]])

    # Initialize and train the neural network with momentum
    nn_with_momentum = NeuralNetwork(input_size=1, hidden_size=2, output_size=1, learning_rate=0.1,
momentum=0.9)
    print("\nTraining with momentum:")
    nn_with_momentum.train(X, y, epochs=10000)
```

```
Training without momentum:
Epoch 0, Loss: 0.2708
Epoch 1000, Loss: 0.0547
Epoch 2000, Loss: 0.0135
Epoch 3000, Loss: 0.0067
Epoch 4000, Loss: 0.0042
Epoch 5000, Loss: 0.0031
Epoch 6000, Loss: 0.0024
Epoch 7000, Loss: 0.0019
Epoch 8000, Loss: 0.0016
Epoch 9000, Loss: 0.0014

Training with momentum:
Epoch 0, Loss: 0.2514
Epoch 1000, Loss: 0.0014
Epoch 2000, Loss: 0.0006
Epoch 3000, Loss: 0.0004
Epoch 4000, Loss: 0.0002
Epoch 5000, Loss: 0.0002
Epoch 6000, Loss: 0.0002
Epoch 7000, Loss: 0.0001
Epoch 8000, Loss: 0.0001
Epoch 9000, Loss: 0.0001
```

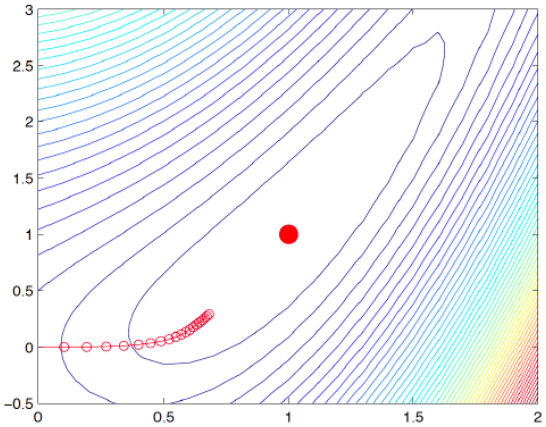
Adam

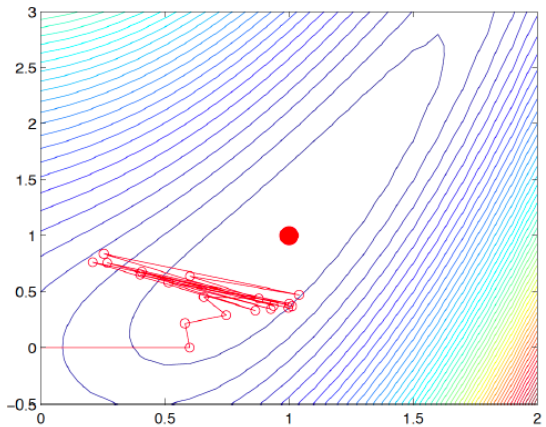
- *Adaptive Moment Estimation (Adam)*
  - Adam computes a **weighted average of past gradients** (first moment of the gradient)
  - Adam also computes a **weighted average of past squared gradients** (second moment of the gradient)
- Other commonly used optimization methods include:
  - Adagrad, Adadelta, RMSprop, Nadam, etc.
  - **Most commonly used optimizers nowadays are Adam and SGD with momentum**

Learning Rate

- *Learning rate*
  - The gradient tells us the direction in which the loss has the steepest rate of increase, but it does not tell us how far along the opposite direction we should step
  - Choosing the learning rate (also called the step size) is one of the most important hyper-parameter settings for NN training

LR too small

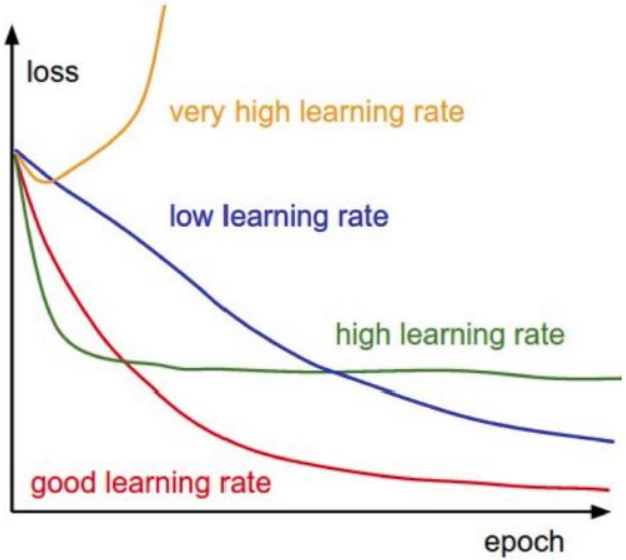




LR too large

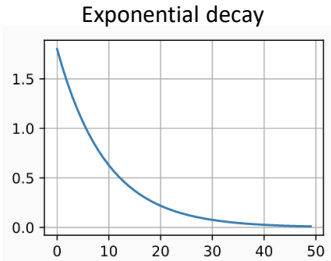
Learning Rate

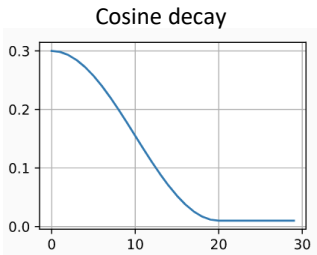
- Training loss for different learning rates
  - High learning rate: the loss increases or plateaus too quickly
  - Low learning rate: the loss decreases too slowly (takes many epochs to reach a solution)

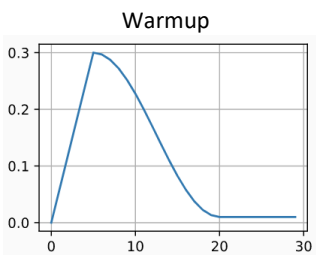


Learning Rate Scheduling

- *Learning rate scheduling* is applied to change the values of the learning rate during the training
  - *Annealing* is reducing the learning rate over time (a.k.a. learning rate decay)
    - Approach 1: reduce the learning rate by some factor every few epochs
      - Typical values: reduce the learning rate by a half every 5 epochs, or divide by 10 every 20 epochs
    - Approach 2: exponential or cosine decay gradually reduce the learning rate over time
    - Approach 3: reduce the learning rate by a constant (e.g., by half) whenever the validation loss stops improving (split data to training set and validation set)
      - In TensorFlow: `tf.keras.callbacks.ReduceLROnPlateau()`
        - Monitor: validation loss, factor: 0.1 (i.e., divide by 10), patience: 10 (how many epochs to wait before applying it), Minimum learning rate: 1e-6 (when to stop)
  - *Warmup* is gradually increasing the learning rate initially, and afterward let it cool down until the end of the training







Learning Rate Scheduling

Refer: Barebones PyTorch Backpropagation

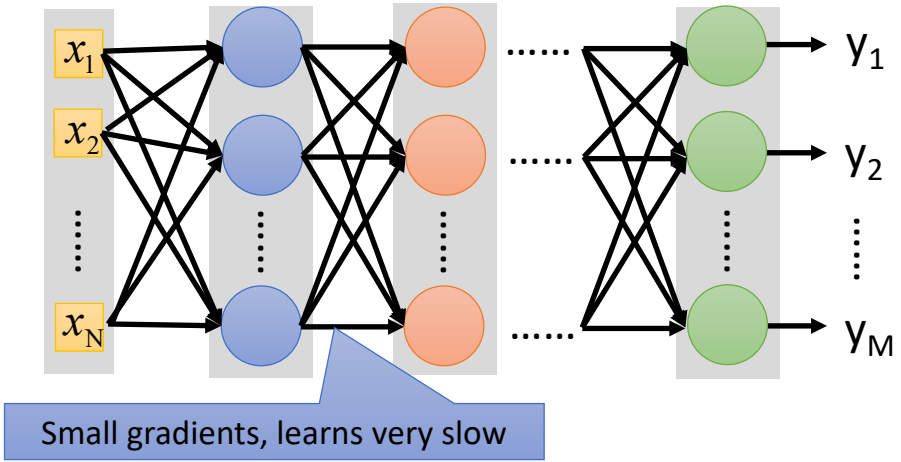
```
optimizer = torch.optim.SGD(network.parameters(), lr=0.1)
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)

for epoch in range(epochs):
    for x, y in zip(X, Y):
        optimizer.zero_grad()          # Reset Gradients
        y_pred = network(x)
        loss = criterion(y_pred, y)
        loss.backward()                # Backpropagate, calculate gradients
        optimizer.step()               # Update Weights with LR
    scheduler.step()                   # Reduce LR for each epoch


# Note that scheduler is per epoch.
# What happens if it's called in the inner loop?
```

Vanishing Gradient Problem

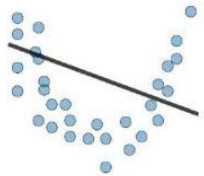
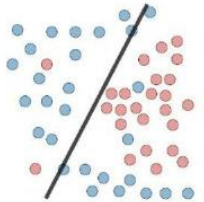
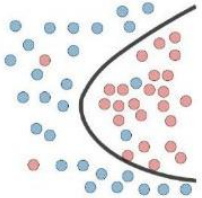
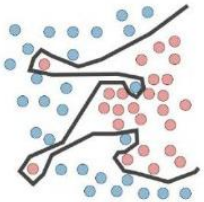
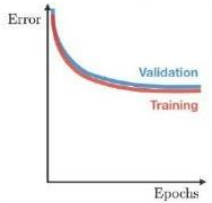
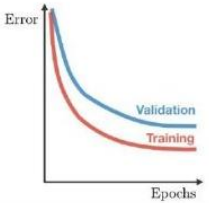
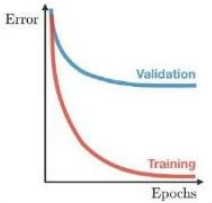
- In some cases, during training, the gradients can become either very small (vanishing gradients) of very large (exploding gradients)
  - They result in very small or very large update of the parameters
  - Solutions: ReLU activations, batch normalization



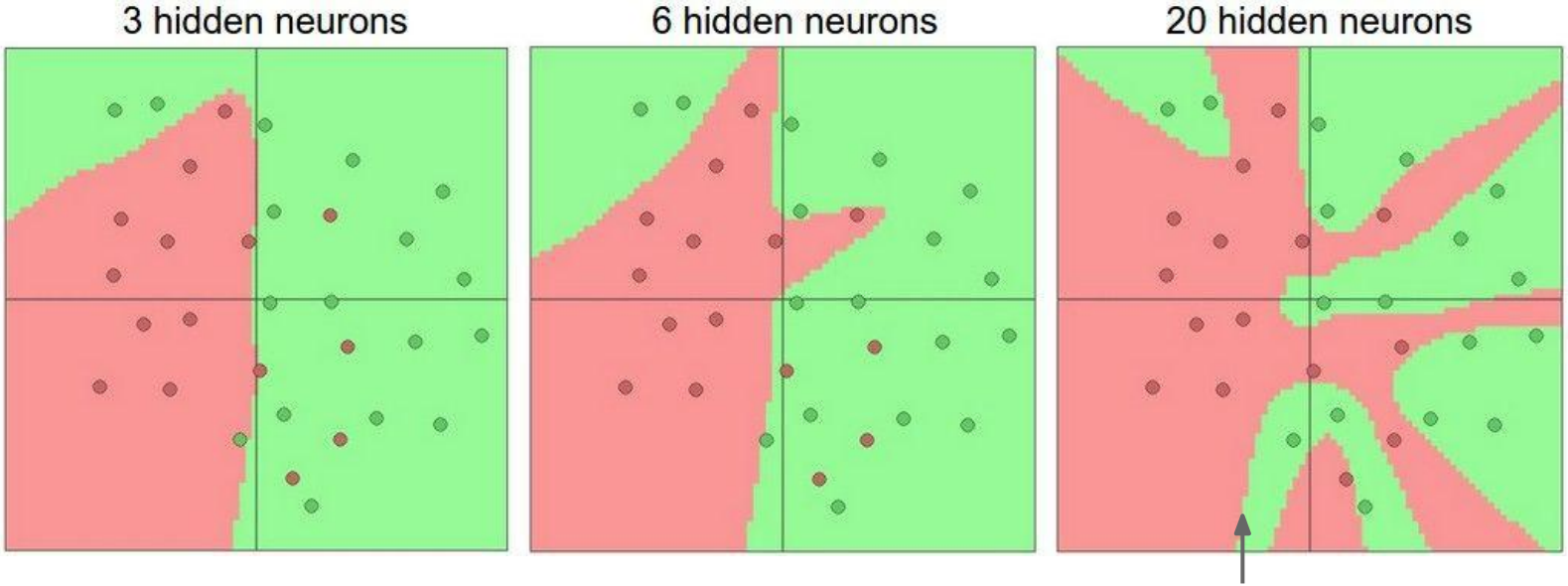
Generalization

- *Underfitting*
  - The model is too "simple" to represent all the relevant target characteristics
    - e.g., model with too few parameters
  - Produces high error on the training set and high error on the validation set

- *Overfitting*
  - The model learns patterns specific to training data, including noise, reducing generalization to unseen data
    - e.g., model with too many parameters
  - Produces low error on the training set but high error on the validation set

	Underfitting	Just right	Overfitting
Symptoms	- High training error - Training error close to test error - High bias	- Training error slightly lower than test error	- Low training error - Training error much lower than test error - High variance
Regression			
Classification			
Deep learning			
Remedies	- Complexify model - Add more features - Train longer		- Regularize - Get more data

Setting the number of layers and their sizes: Classification



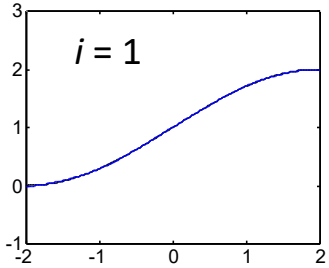
Choice of Architecture: Regression

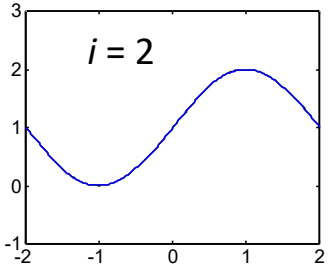
Target function: $g(p) = 1 + \sin\left(\frac{i\pi}{4}p\right)$

In the figures, **the blue line** is the neural network output, while **the black line** is the target function.

1-3-1 Network

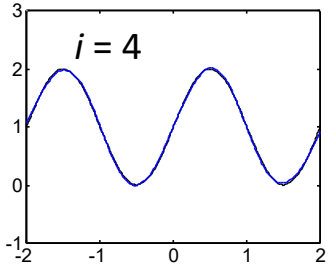
OK for this Network

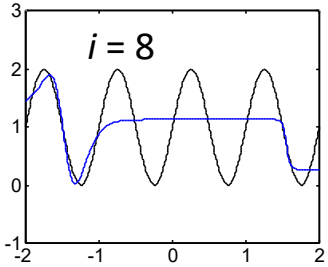




OK for this Network

OK for this Network





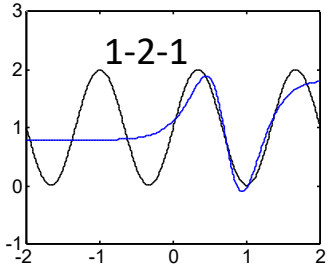
Too complicated for this Network

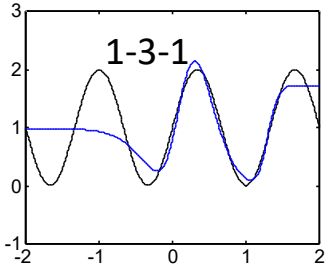
Choice of Network Architecture: Regression

Target function: $g(p) = 1 + \sin\left(\frac{i\pi}{4}p\right)$

In the figures, **the blue line** is the neural network output, while **the black line** is the target function.

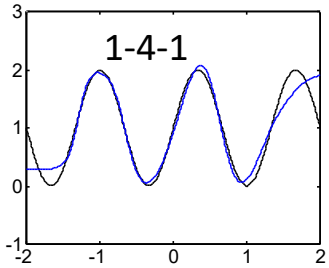
Not enough hidden neurons

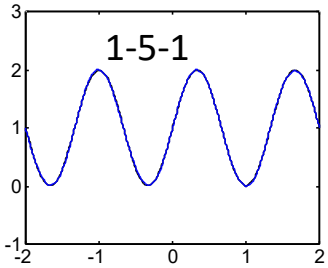




Not enough hidden neurons

Not enough hidden neurons





Enough hidden neurons

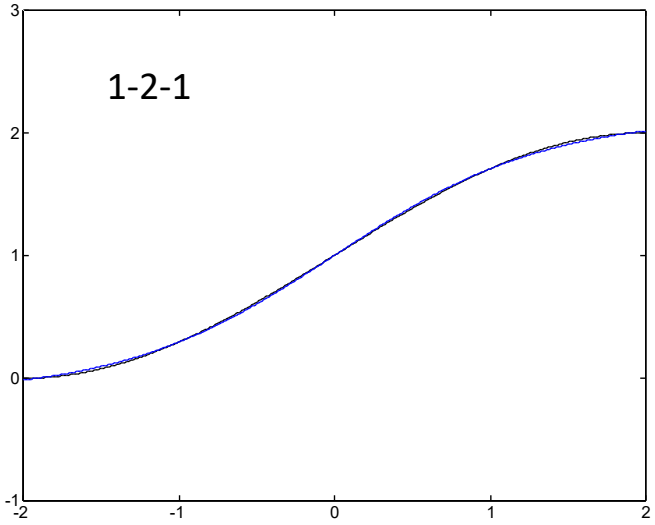
Generalization: Regression

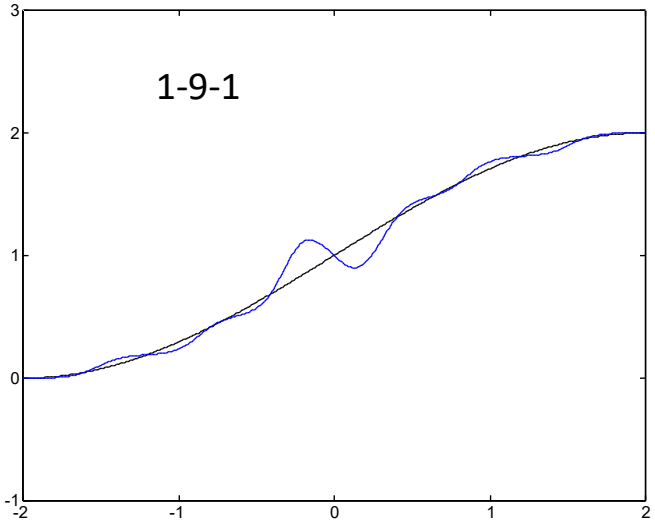
$$\{\mathbf{p}_1, \mathbf{t}_1\}, \{\mathbf{p}_2, \mathbf{t}_2\}, \ldots, \{\mathbf{p}_Q, \mathbf{t}_Q\}$$

Target output/function: $g(p) = 1 + \sin\left(\frac{\pi}{4}p\right)$ $\quad p = -2, -1.6, -1.2, \ldots, 1.6, 2$

In the figures, **the blue line** is the neural network output, while **the black line** is the target function.

Enough hidden neurons





Too many hidden neurons

Regularization: ℓ_2 *regularization*

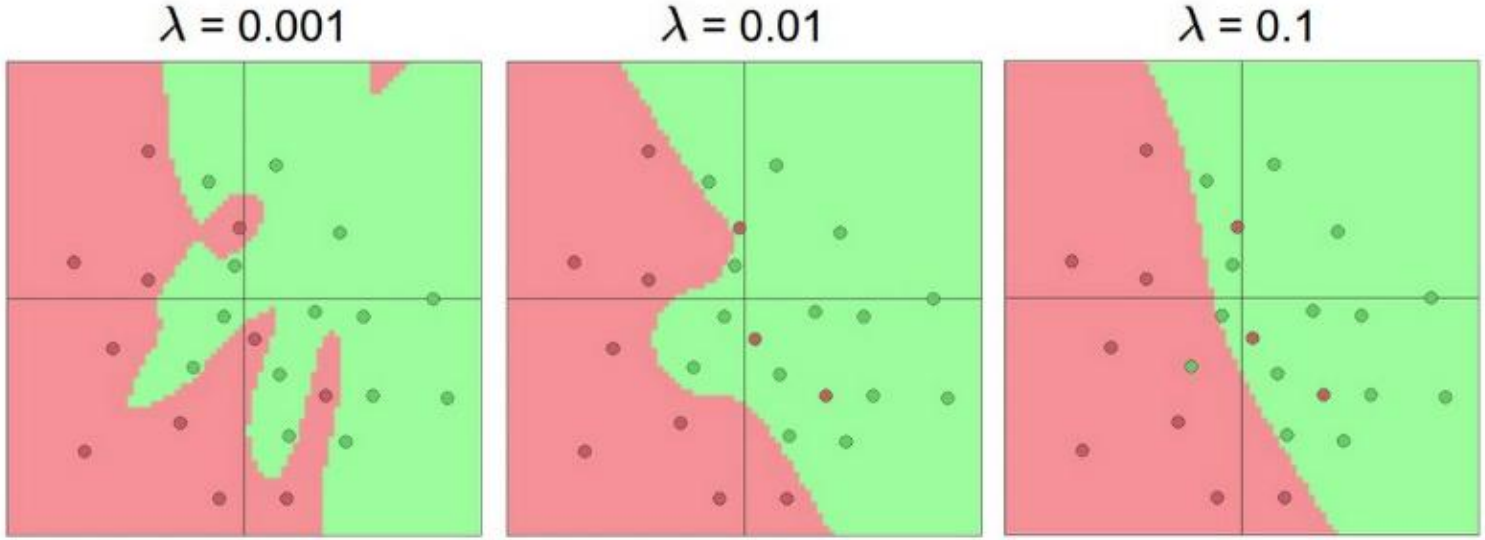
- A regularization term that penalizes large weights is added to the loss function

$$\mathcal{L}_{reg}(\theta) = \underbrace{\mathcal{L}(\theta)}_{\text{Data loss}} + \underbrace{\lambda \sum_k \theta_k^2}_{\text{Regularization loss}}$$

- The regularization coefficient λ determines how dominant the regularization is during the gradient computation

Regularization: Effect of the regularization coefficient λ

- Large regularization coefficient → penalty for weights with large values



Regularization: ℓ_1 *regularization*

- The regularization term is based on the ℓ_1 norm of the weights

$$\mathcal{L}_{reg}(\theta) = \mathcal{L}(\theta) + \lambda \sum_k |\theta_k|$$

- ℓ_1 regularization is less common with NN
  - Often performs worse than ℓ_2 regularization
- It is also possible to combine ℓ_1 and ℓ_2 regularization
  - Called elastic net regularization

$$\mathcal{L}_{reg}(\theta) = \mathcal{L}(\theta) + \lambda_1 \sum_k |\theta_k| + \lambda_2 \sum_k \theta_k^2$$

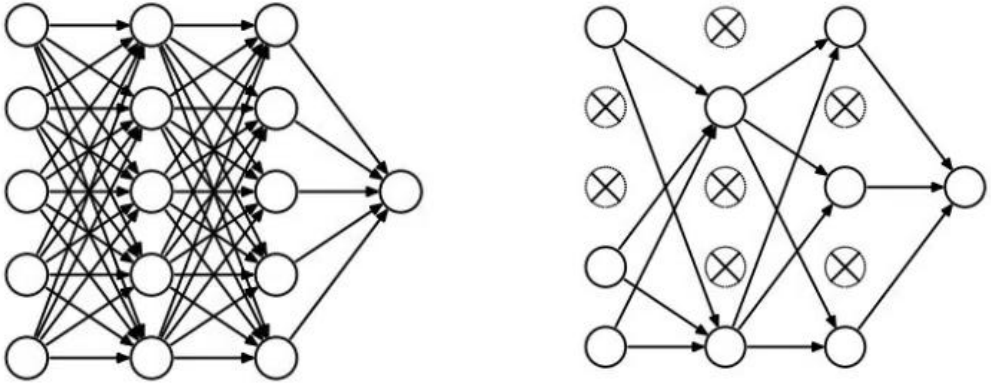
Regularization in PyTorch

- Regularization is implemented **within** the optimizer.
https://pytorch.org/docs/stable/generated/torch.optim.SGD.html

```
# 0.01 L2 Regularization
optimizer = torch.optim.SGD(..., weight_decay=0.01)
```

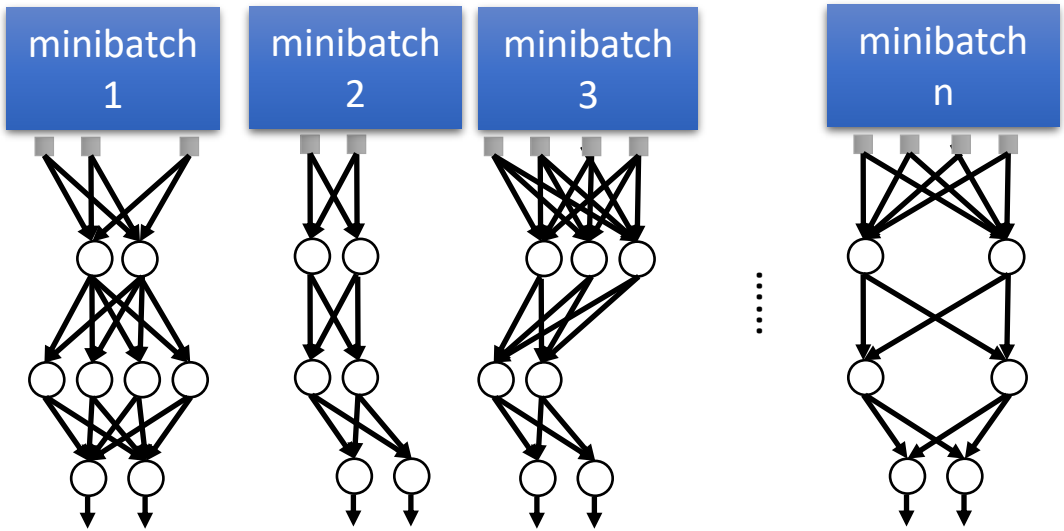
Regularization: Dropout

- Randomly drop units (along with their connections) during training
- Each unit is dropped with a fixed dropout rate p, independent of other units
- The hyper-parameter p needs to be chosen (tuned)
  - Often, between 20% and 50% of the units are dropped



Regularization: Dropout

- Dropout is a kind of ensemble learning
  - Use one mini-batch to train one network with dropout, each time resulting a different architecture



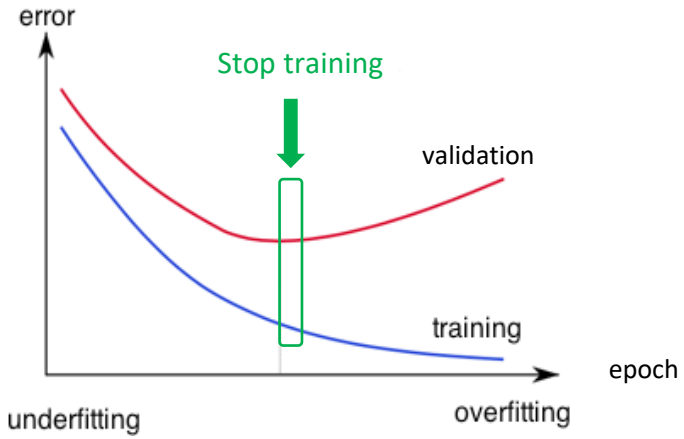
Regularization: Dropout

```
# Define network with dropout of probability 0.5
>>> network = torch.nn.Sequential(
    torch.nn.Linear(2, 2),
    torch.nn.Dropout(p=0.5),
)
# Network will randomly drop connections, thus, we get different outputs for the same input.
>>> network(torch.tensor([0, 1], dtype=torch.float))
[0.6059, 0.0040],
>>> network(torch.tensor([0, 1], dtype=torch.float))
[0., 0.]

# We should disable dropout during evaluation.
>>> network.eval()
[0.3030, 0.0020]
>>> network.eval()
[0.3030, 0.0020]
```

Regularization: Early Stopping

- During model training, use a validation set
  - E.g., validation/train ratio of about 25% to 75%
- Stop when the validation accuracy (or loss) has not improved after n epochs
  - The parameter n is called patience



Batch Normalization

- ***Batch normalization layers*** act similarly to the data preprocessing steps mentioned earlier
  - They calculate the mean μ and variance σ of a batch of input data, and normalize the data x to a zero mean and unit variance
  - i.e., $\hat{x} = \frac{x-\mu}{\sigma}$
- BatchNorm layers alleviate the problems of proper initialization of the parameters and hyper-parameters
  - Normalizes the activations, making them less dependent on the scale of the parameters in the initialization process
    - Less likely to suffer from vanishing or exploding gradients during training
  - Reduces internal covariate shift, which is the change in the distribution of activations due to parameter updates during training
    - Stabilizes the training process and allows for higher learning rates without the risk of divergence. Consequently, models converge faster and more reliably
  - BatchNorm layers are very common with convolutional NNs

Batch Normalization

Batch Normalization is just another layer in PyTorch

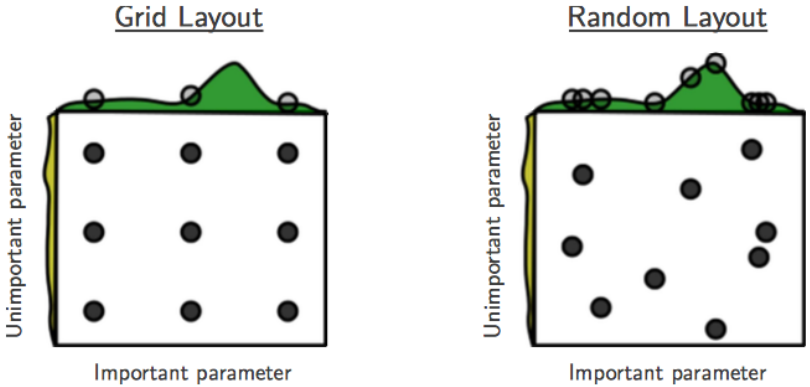
```
network = torch.nn.Sequential(
    torch.nn.Linear(2, 2),
    torch.nn.BatchNorm1d(2),
    torch.nn.Linear(2, 2),
)
```

Hyper-parameter Tuning

- Training NNs can involve setting many ***hyper-parameters***:
  - Number of layers, and number of neurons per layer
  - Initial learning rate
  - Learning rate decay schedule (e.g., decay constant)
  - Optimizer type
  - Regularization parameters (ℓ_2 penalty, dropout rate)
  - Batch size
  - Activation functions
  - Loss function
- Hyper-parameter tuning can be time-consuming for larger NNs

Hyper-parameter Tuning

- Grid search
  - Check all values in a range with a step value
- Random search
  - Randomly sample values for the parameters
  - Often preferred to grid search
- Bayesian hyper-parameter optimization
  - Is an active area of research

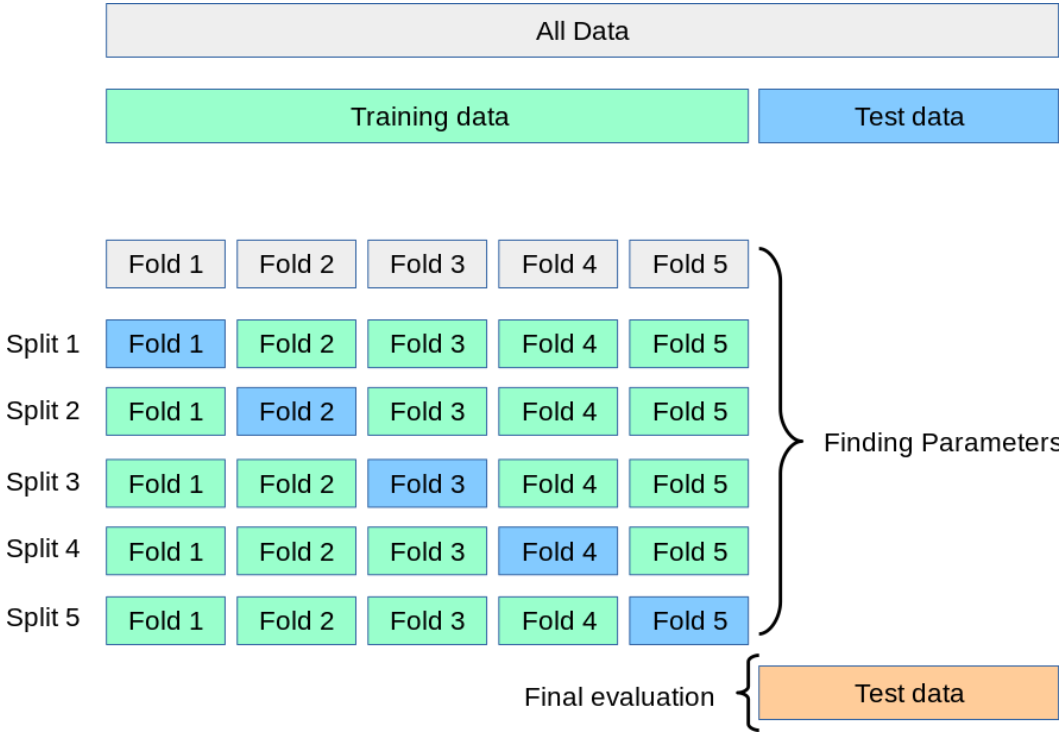


k-Fold Cross-Validation

- Using ***k-fold cross-validation*** for hyper-parameter tuning is common when the size of the training data is small
  - It also leads to a better and less noisy estimate of the model performance by averaging the results across several folds
- e.g., 5-fold cross-validation (see the figure on the next slide)
  1. Split the data into train data and test data
  2. Split the train data into 5 equal folds
  3. First use folds 2-5 for training and fold 1 for validation
  4. Repeat by using fold 2 for validation, …, then fold 3, fold 4, and fold 5
  5. Average the results over the 5 runs (for reporting purposes)
  6. Once the best hyper-parameters are determined, evaluate the model on the test data

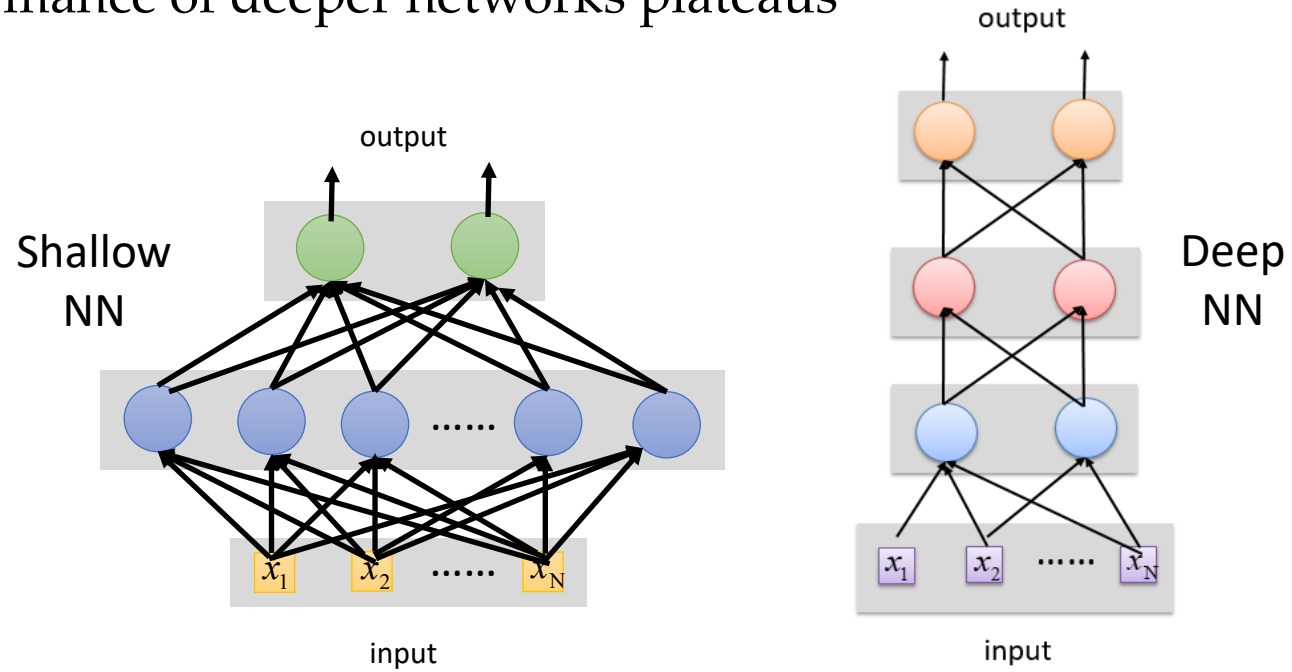
k-Fold Cross-Validation

- Illustration of a 5-fold cross-validation



Deep vs Shallow Networks

- Deeper networks perform better than shallow networks
  - But only up to some limit: after a certain number of layers, the performance of deeper networks plateaus



MLP to Classify Iris (1 hidden layer, 100 hidden neurons)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Splitting the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initializing MLP classifier
mlp_clf = MLPClassifier(hidden_layer_sizes=(100,), activation='relu', solver='adam', max_iter=1000, random_state=42)

# Training the classifier
mlp_clf.fit(X_train, y_train)

# Predictions on the testing set
predictions = mlp_clf.predict(X_test)

# Accuracy calculation
accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

MLP to Fit a Curve (1 hidden layer, 100 hidden neurons)

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor

# Generate some sample data
np.random.seed(0)
X = np.linspace(0, 10, 100).reshape(-1, 1)  # Input features
y = np.sin(X).ravel() + np.random.normal(0, 0.1, 100)  # Target values with added noise

# Initialize MLP regressor
mlp_regressor = MLPRegressor(hidden_layer_sizes=(100,), activation='relu', solver='adam', max_iter=1000, random_state=42)

# Fit the MLP regressor to the data
mlp_regressor.fit(X, y)

# Generate predictions using the trained MLP regressor
predictions = mlp_regressor.predict(X)

# Plot the original data and the fitted curve
plt.scatter(X, y, color='blue', label='Original Data')
plt.plot(X, predictions, color='red', label='Fitted Curve')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Fitting a Curve using MLP')
plt.legend()
plt.show()
```